

微型计算机原理

第二章 16位微处理器8086系统结构



刘剑锋



2025年2月20日星期四

第二章 16位微处理器8086系统结构

8086系统结构

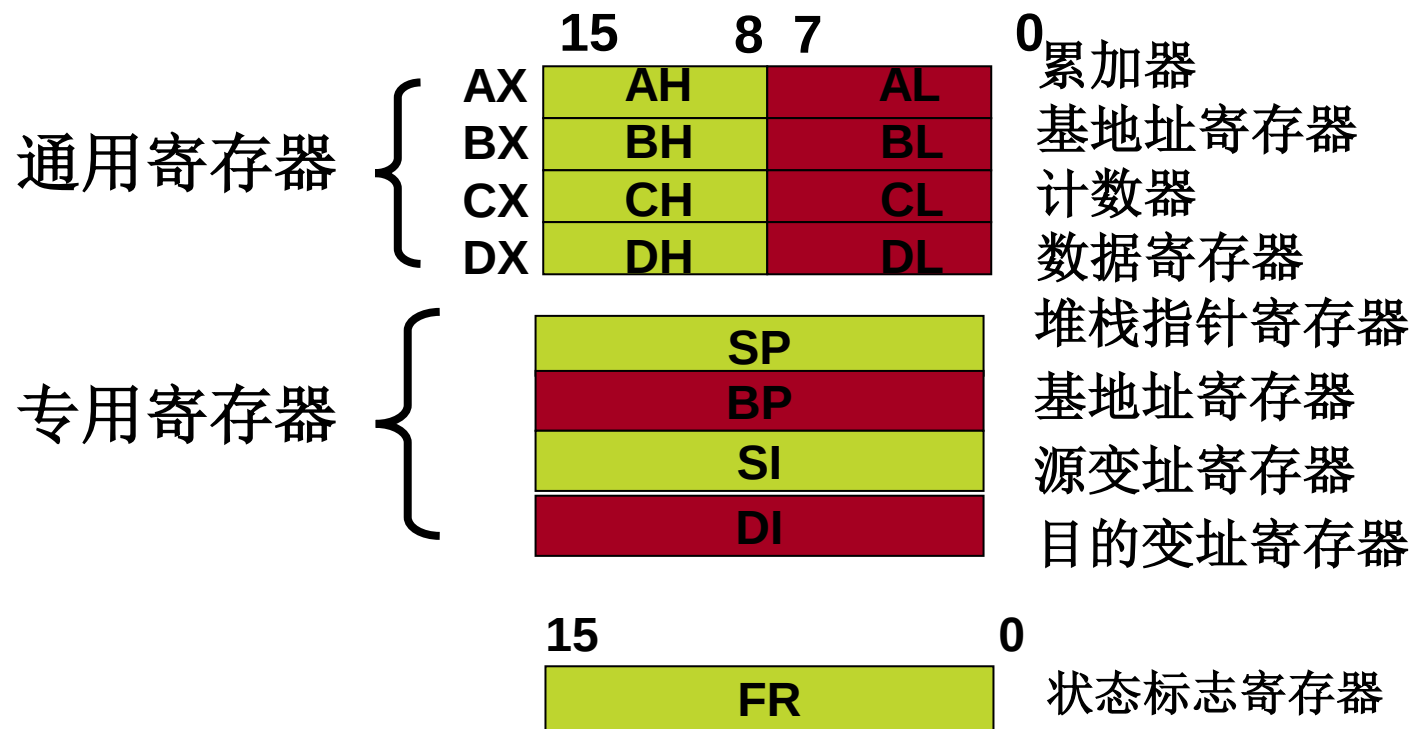


- 
- **8086**内部由两部分组成:

执行单元 (**EU**)

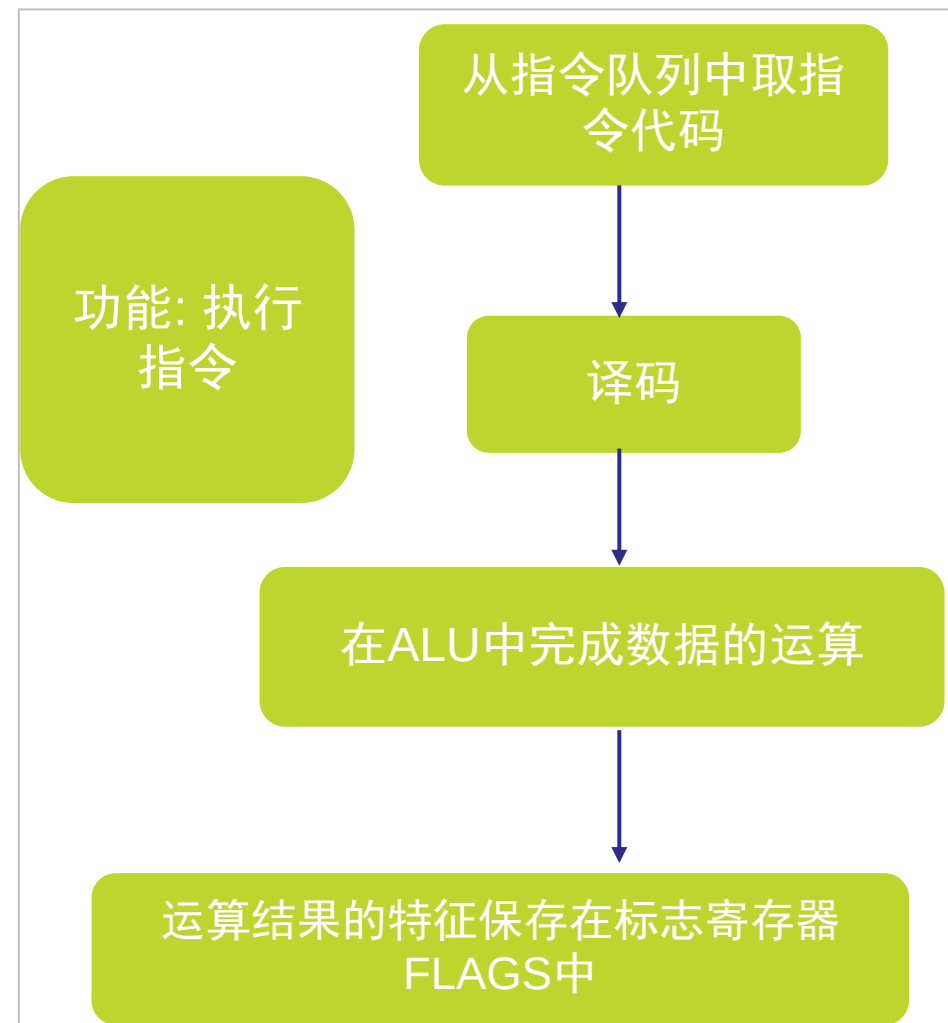
总线接口单元 (**BIU**)

2.1.1 执行部件 (Execution Unit)



算术逻辑单元ALU

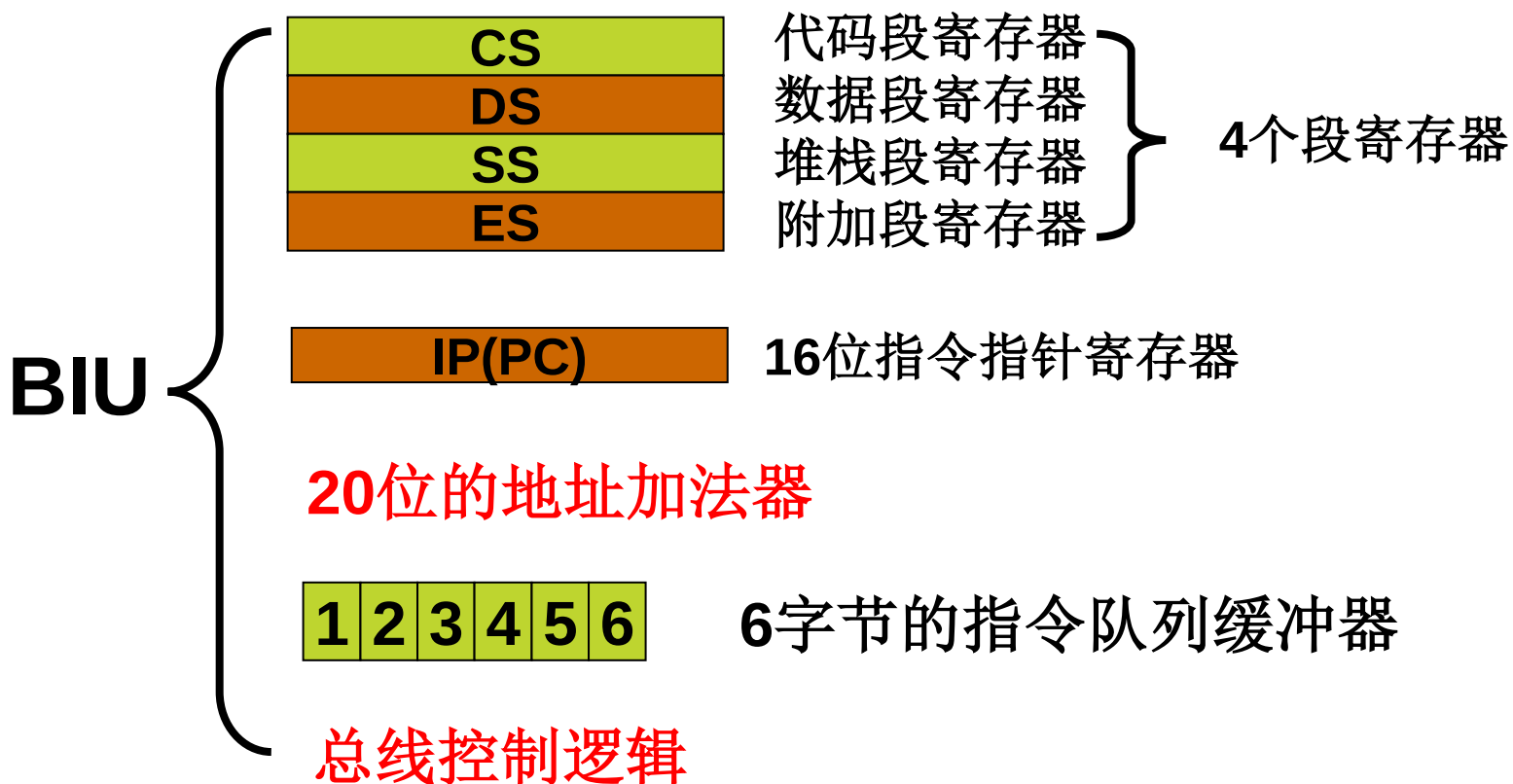
EU控制器



2.1.2 总线接口部件 (Bus Interface Unit)

功能:

- (1) 从内存取指令送到指令队列。
- (2) CPU执行指令时, 到指定的位置取操作数, 并将其送至要求的位置单元中。

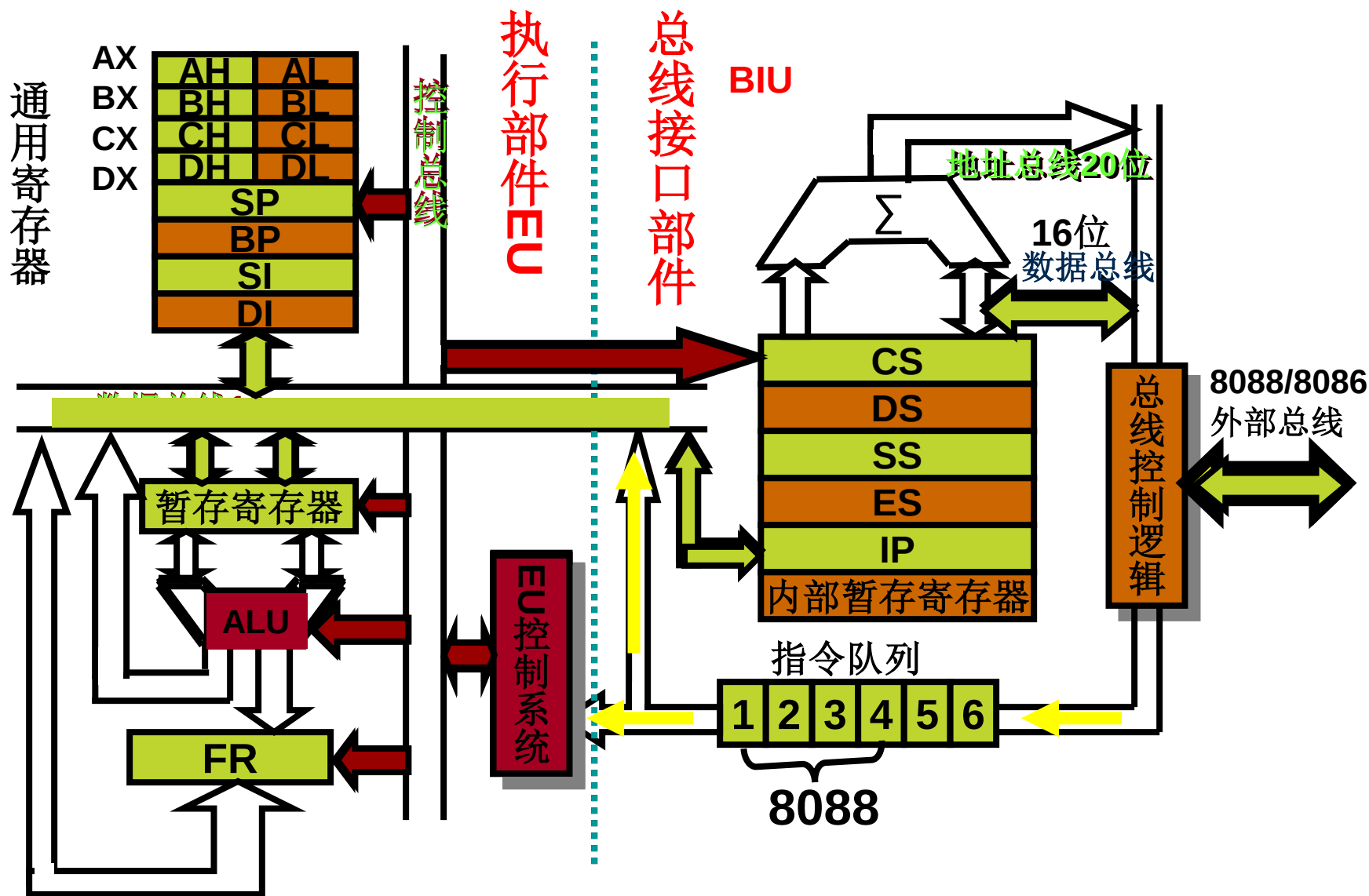


说明:

(1) **指令队列缓冲器**: 在执行指令的同时, 将取下一条指令, 并放入指令队列缓冲器中。CPU执行完一条指令后, 可以执行下一条指令 (流水线技术)。提高CPU效率。

(2) **地址加法器**: 产生20位地址。CPU内无论是段地址寄存器还是偏移量都是16位的, 通过地址加法器产生20位地址。

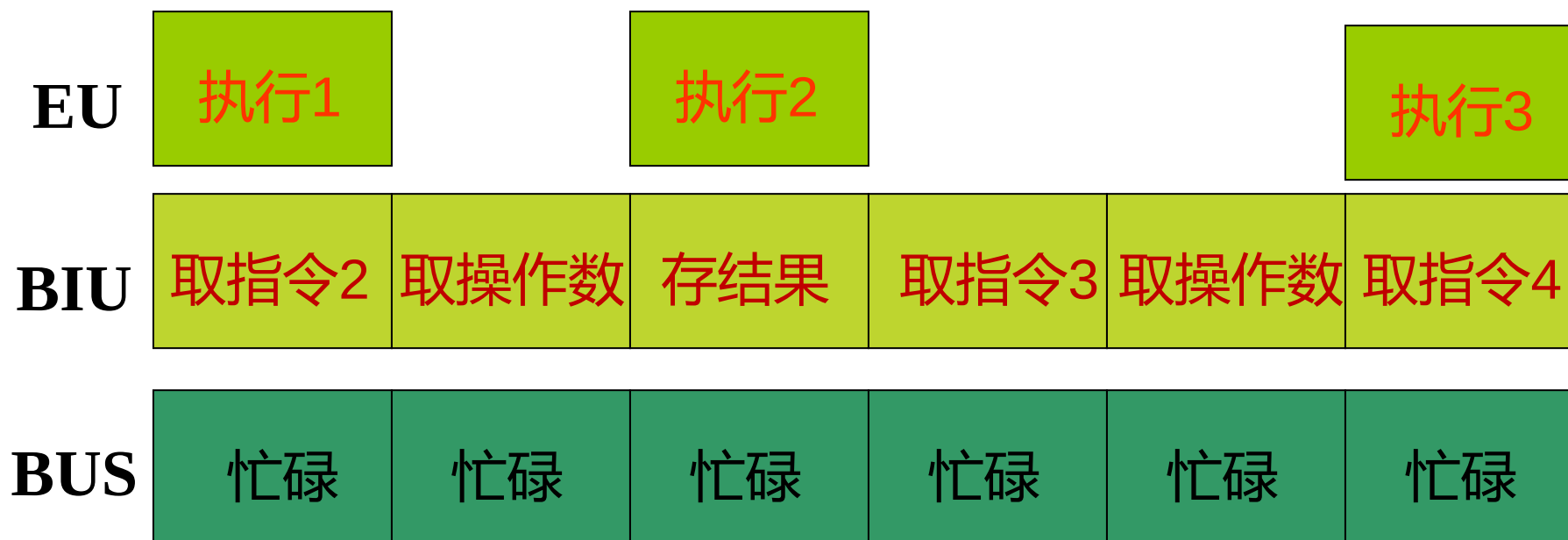
8088/8086的内部结构图



➤ 串行工作方式



➤ 8086CPU采用并行工作方式



BIU和EU是并行工作的,按流水线技术原则管理

1. 当8086指令队列中有**两个**空字节时,BIU自动把指令取到队列中。
2. EU从指令队列取指、执行。执行过程中如果要访问存储器或I/O,而此时, BIU正在取指, 完成取指后, 响应EU的总线请求。
3. 指令队列已满, EU又没有总线访问, BIU进入空闲状态。
4. 执行转移、调用和返回指令时, 指令队列中的原有内容自动消除, BIU往指令队列中装入另一程序段中的指令。



2.2 8086/8088的内部寄存器



8086/8088的内部含14个16位寄存器，按功能可分为三类

➤8个通用寄存器

数据寄存器 (AX, BX, CX, DX)

地址指针寄存器 (SP, BP)

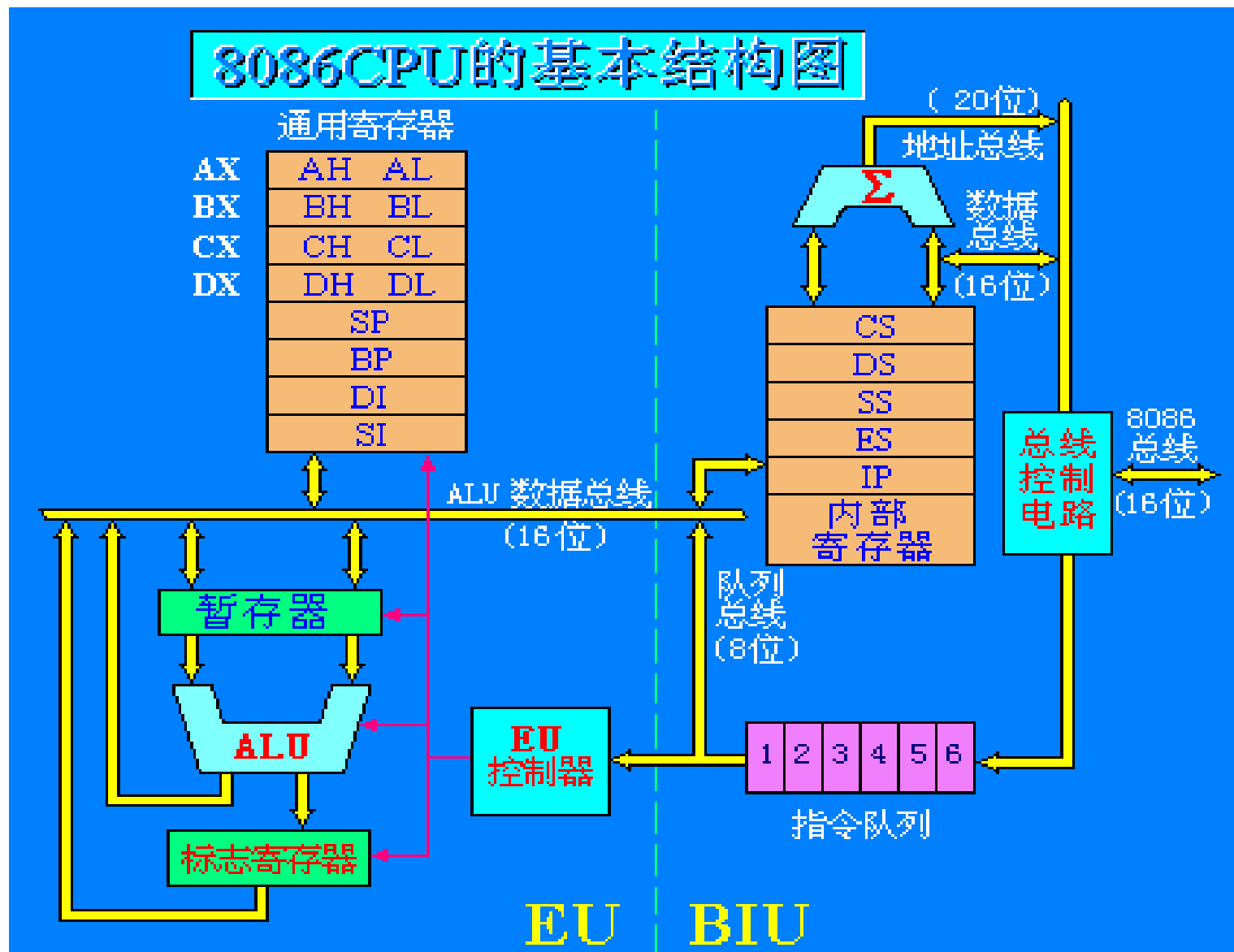
变址寄存器 (SI, DI)

➤4个段寄存器

CS, DS, SS, ES

➤1个标志寄存器FG

1个指令指针寄存器IP



数据寄存器

- 8086含4个16位数据寄存器，它们又可分为8个8位寄存器，即：

- AX	→	AH, AL
- BX	→	BH, BL
- CX	→	CH, CL
- DX	→	DH, DL

数据寄存器特有的习惯用法

AX：累加器。多用于存放中间运算结果。所有 I/O指令必须都通过AX与接口传送信息；

BX：基址寄存器。在间接寻址中用于存放基地址；

CX：计数寄存器。用于在循环或串操作指令中存放循环次数或重复次数；

DX：数据寄存器。在32位乘除法运算时，存放高16位数；在间接寻址的I/O指令中存I/O端口地址。

地址指针寄存器

- SP: **堆栈指针寄存器**，其内容为栈顶的偏移地址；
- BP: **基址指针寄存器**，常用于在访问内存时存放内存单元的偏移地址。

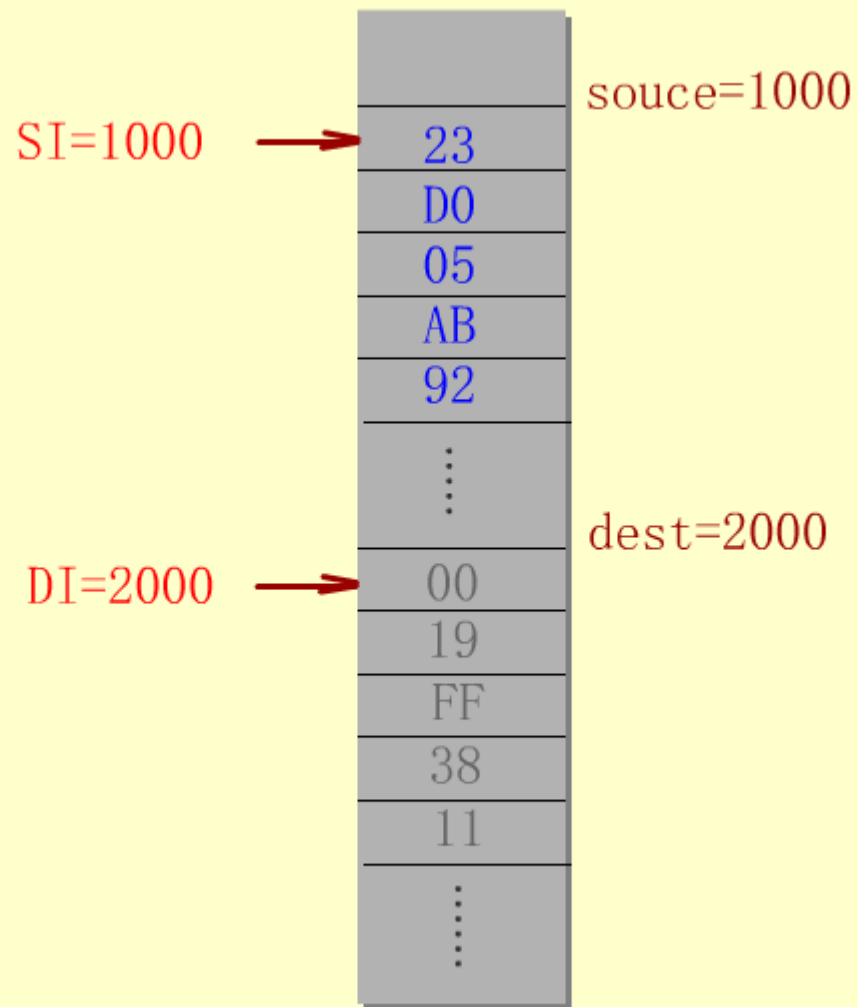
BX与BP在应用上的区别

- 作为通用寄存器，二者均可用于存放数据；
- 作为基址寄存器：
 - BX**通常用于寻址数据段，与**DS**或ES搭配使用
 - BP**则通常用于寻址堆栈段，与**SS**搭配使用

变址寄存器

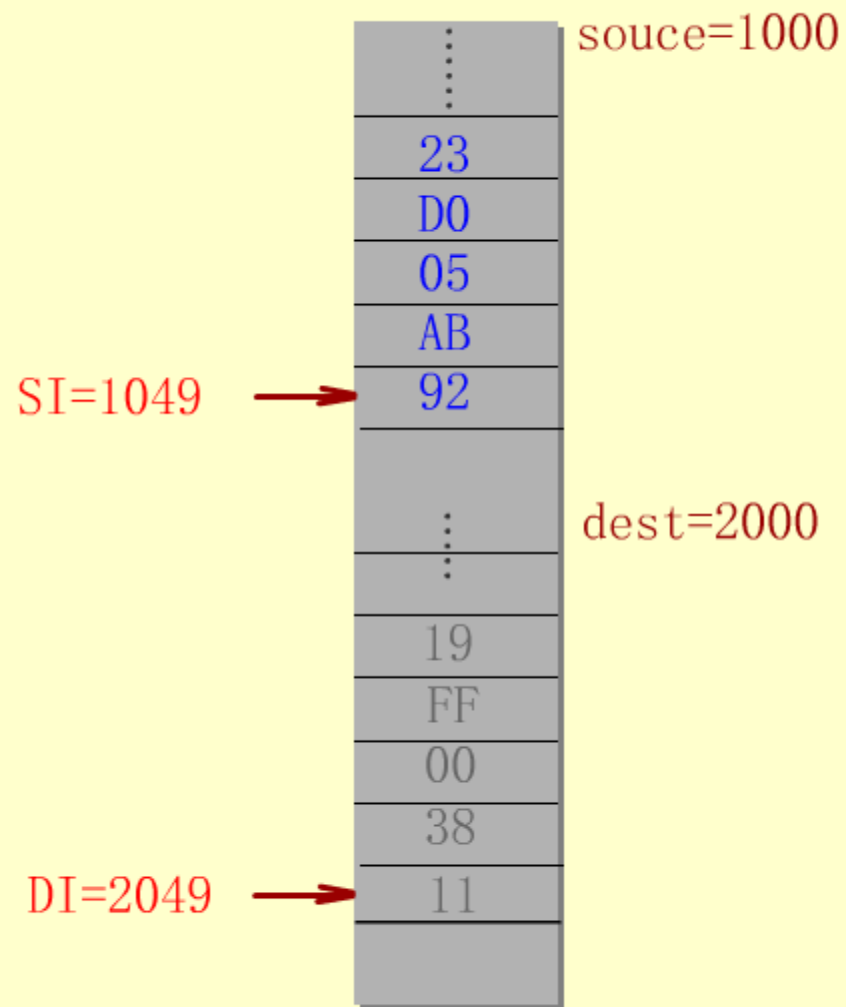
- SI：源变址寄存器
- DI：目标变址寄存器
- 变址寄存器常用于指令的间接寻址或变址寻址。特别是在串操作指令中，用SI存放源操作数的偏移地址，而用DI存放目标操作数的偏移地址。

串传送 MOVSB (正向 DF=0)



播放

串传送 MOVSW (反向 DF=1)



播放

2. 段寄存器

用于存放逻辑段的段基地址 (逻辑段的概念后面将要介绍)

CS: 代码段寄存器

代码段用于存放指令代码

DS: 数据段寄存器

ES: 附加段寄存器

数据段和附加段用来存放操作数

SS: 堆栈段寄存器

堆栈段用于存放返回地址, 保存寄存器内容, 传递参数

3. 控制寄存器

- IP: 指令指针寄存器, 其内容为下一条要执行的指令的偏移地址
 - FLAGS: 标志寄存器
 - 状态标志: 存放运算结果的特征
 - 控制标志: 控制某些特殊操作
- { 6个状态标志位 (CF, SF, AF, PF, OF, ZF)
3个控制标志位 (IF, TF, DF)

标志寄存器

16位寄存器，其中有7位未用。

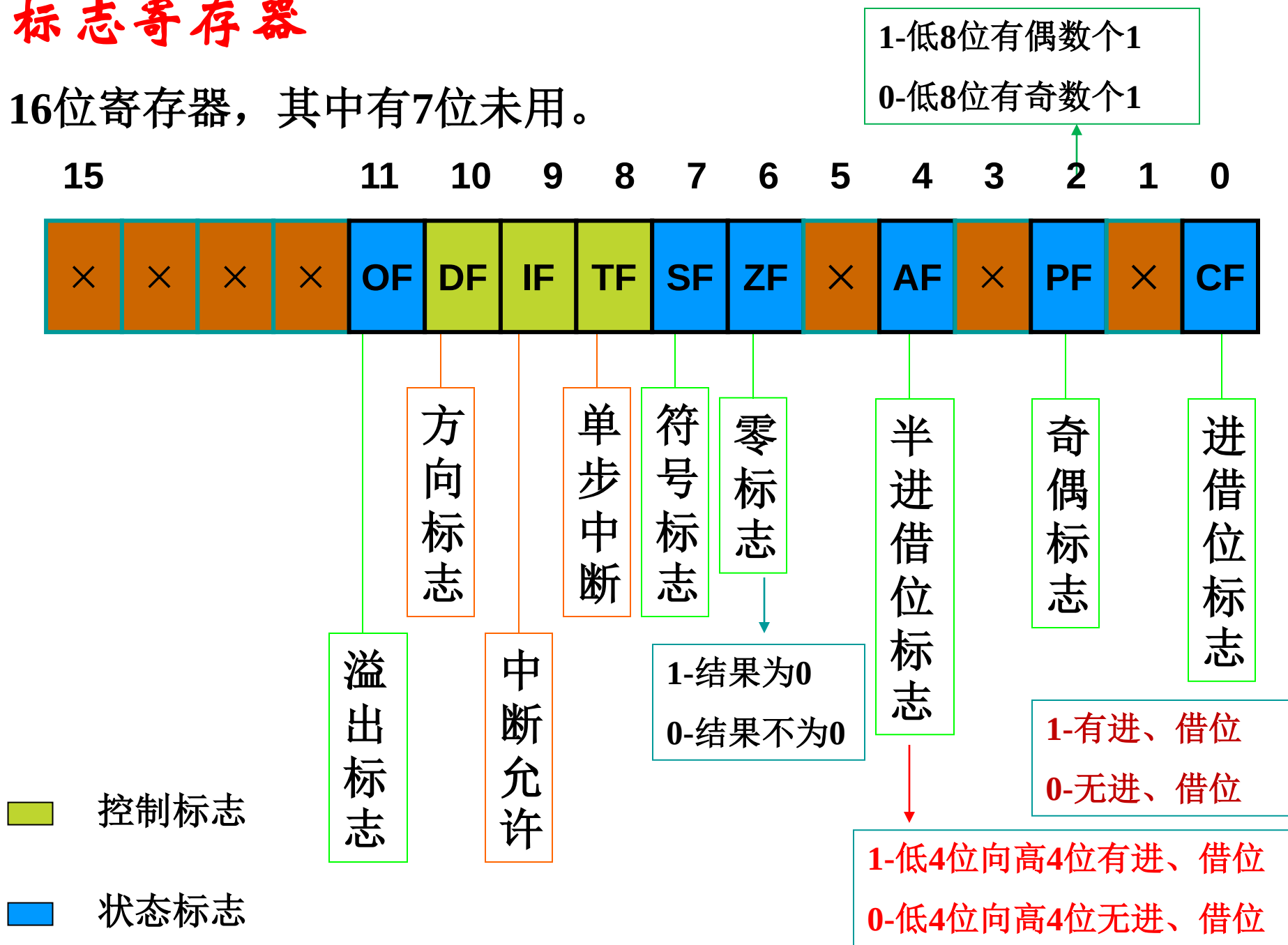


表 8086 CPU 标志位情况

名称	符号	功能
符号标志	SF	与运算结果的最高位相同，当数据用补码表示时，负数的最高位为1，所以符号标志表示运算执行后的结果是正还是负
零标志	ZF	当前的运算结果为零，当前的运算结果为非零
奇偶标志	PF	运算结果所含的1的个数为偶数
进位标志	CF	当执行一个加法运算使最高位产生进位时，或者执行一个减法运算引起最高位产生借位时，此外，循环指令也影响这一标志
辅助进位标志	AF	加法运算时，如果第3位往第4位有进位；减法运算时，如果第3位往第4位有借位。辅助进位标志一般在BCD码运算中作为是否进行十进制调整的判断依据
溢出标志	OF	运算过程中产生溢出时，所谓溢出，是指当字节运算的结果超出了范围 -128~ +127，或者当字运算的结果超出了范围 -32768~ +32767时称为溢出
方向标志	DF	控制串操作指令用的标志。DF=0, 串操作过程中的地址会不断增值；DF=1, 串操作过程中的地址会不断减值
中断标志	IF	控制可屏蔽中断的标志。IF=0, CPU不能对可屏蔽中断请求作出响应；IF=1, CPU可以接受可屏蔽中断请求
跟踪标志	TF	CPU按跟踪方式执行指令

例:将5394H与-777FH相加,并说明其标志位的状态。

【解】先求-777FH的补码:

原码: 1111 0111 0111 1111

反码: 1000 1000 1000 0000

补码: 1000 1000 1000 0001

再做加法运算: 0101 0011 1001 0100

+ 1000 1000 1000 0001

1101 1100 0001 0101

结果标志位为: CF=0、PF=0、AF=0

ZF=0、SF=1、OF=0



2.3 8086微处理器的引脚信号功能



8086的引脚信号和功能（1）

- 8086是一个40引脚的器件，为了便于组成不同规模的系统，Intel公司为8086设计了两种工作模式。在不同的工作模式下，引脚信号的含义不同。

1. AD15~AD0 地址/数据复用总线，双向，三态
(8088中A15~A8不复用，输出，三态)
2. A19/S6~A16/S3 地址/状态线复用，输出，三态
3. BHE/S7 (8086) 高8位数据总线允许/状态信息复用引脚，输出

S4 S3	当前段寄存器
0 0	使用附加段寄存器
0 1	使用堆栈段寄存器
1 0	使用代码段寄存器
1 1	使用数据段寄存器

GND	01	8086	40	VCC (5V)
AD14	02		39	AD15
AD13	03		38	A16/S3
AD12	04		37	A17/S4
AD11	05		36	A18/S5
AD10	06		35	A19/S6
AD9	07		34	/BHE/S7
AD8	08		33	MN//MX
AD7	09		32	/RD
AD6	10		31	HOLD (/RQ//GT0)
AD5	11		30	HLDA (/RQ//GT1)
AD4	12		29	/WR (/LOCK)
AD3	13		28	M//IO (/S2)
AD2	14		27	DT//R (/S1)
AD1	15		26	/DEN (/S0)
AD0	16		25	ALE (QS0)
NMI	17		24	/INTA (QS1)
INTR	18		23	/TEST
CLK	19		22	READY
GND	20		21	RESET

2.3.1 8086/8088的引脚信号和功能（2）

4. NMI, 非屏蔽中断, 输入 
5. INTR, 可屏蔽中断请求信号, 输入
6. $\overline{RD}$, 读信号, 输出, 三态, 低电平有效
7. CLK, 时钟信号, 输入 
8. RESET, 复位信号, 输入（至少保持4个时钟周期的高电平）
9. READY, 准备好信号, 输入
10. $\overline{TEST}$, 测试信号, 输入, 低电平有效
11. $\overline{MN/MX}$, 最小/最大模式控制引脚, 输入

$\overline{MN/MX} = V_{CC}$ 最小工作方式（单CPU方式）

$\overline{MN/MX} = GND$ 最大工作方式（多CPU方式）

牢记

CPU立即停止操作，清FR，DS，ES，SS、IP及指令队列。同时，置CS为FFFFH。当RESET变为低电平时，CPU从FFFF0H单元开始启动。

12. GND地和 V_{CC} 电源引脚



2.4 8086的存储器组织



8086/8088的存储器结构和堆栈

主要内容：

- 存储器编址
- 存储器分段和物理地址生成
- 堆栈和栈操作指令

数据与指令的存储与访问

内存示意图

所有CPU可以访问的数据与指令都以二进制数的形式存放在内存中

地址是数据存放的门牌号码是标明数据所在位置的唯一代号

地址	内 容
0	00000100B (4)
1	10000000B (128)
2	00010001B (17)
...	
255	00001111B (15)
...	

每个地址空间可以存放8位二进制数

- 
- 8086计算机主存储器是按字节进行组织的，两个相邻的字节被称为一个“字”。
 - 存放的信息若是以字节Byte（8位）为单位的，将在存储器中按顺序排列存放，每个地址单元存放一个字节，即每个字节占用一个地址；
 - 若存放的数据为一个字Word（16位）时，则每一个字占用两个地址相连的存储单元，其中低字节（低8位）存放在低地址中，高字节（高8位）存放在高地址中，并以低地址作为该字的地址。
 - 8086系统只能提供20根地址线来区分内存的不同存储单元，因此8086系统可以连接的内存最多只能包含 2^{20} （1M）字节的存储单元。

一、存储器编址

- 存储器以8位为一个存储单元编址，每一个存储单元用唯一的一个地址码来表示。
- 一个字的两个单元有各自的地址，处于低地址的**字节的地址**为这个**字的地址**
- 在存储器中，任何连续存放的两个字节都可以称为一个字。将偶地址的字称为**规则字**，奇地址的字称为**非规则字**。

例：字2301的地址为0000H，为**规则字**；
字8967的地址为0003H，为**非规则字**。

物理地址	存储器
0000	01
0001	23
0002	45
0003	67
0004	89
0005	AB
0006	CD

存储器中的数据

二、存储器分段和物理地址的生成

- 物理地址
 - **8086**: 20根地址线, 可寻址 2^{20} (1MB)个存储单元
 - CPU送到AB上的20位的地址称为物理地址

物理地址

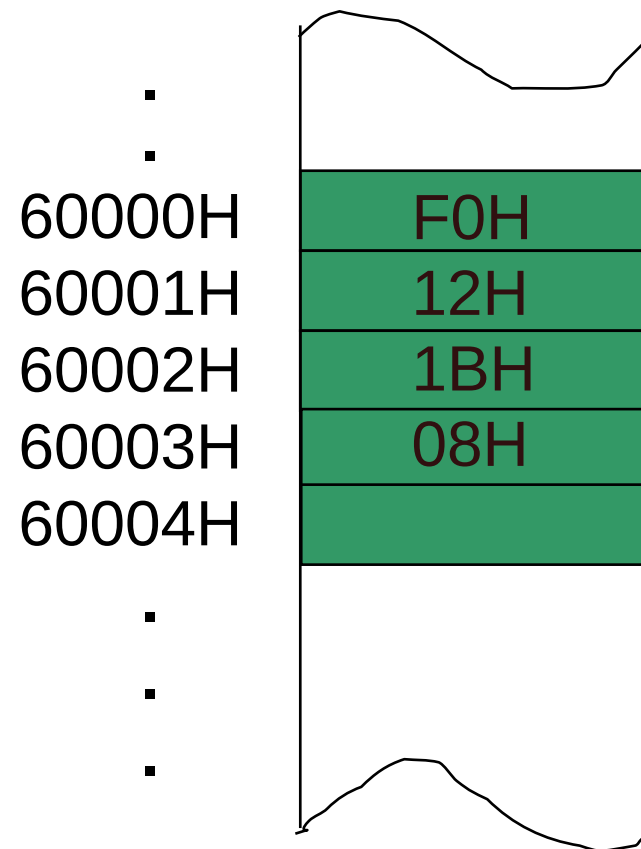
存储器的操作完全基于物理地址。

➤问题：

8086的内部总线和内部寄存器均为16位，如何生成20位地址？

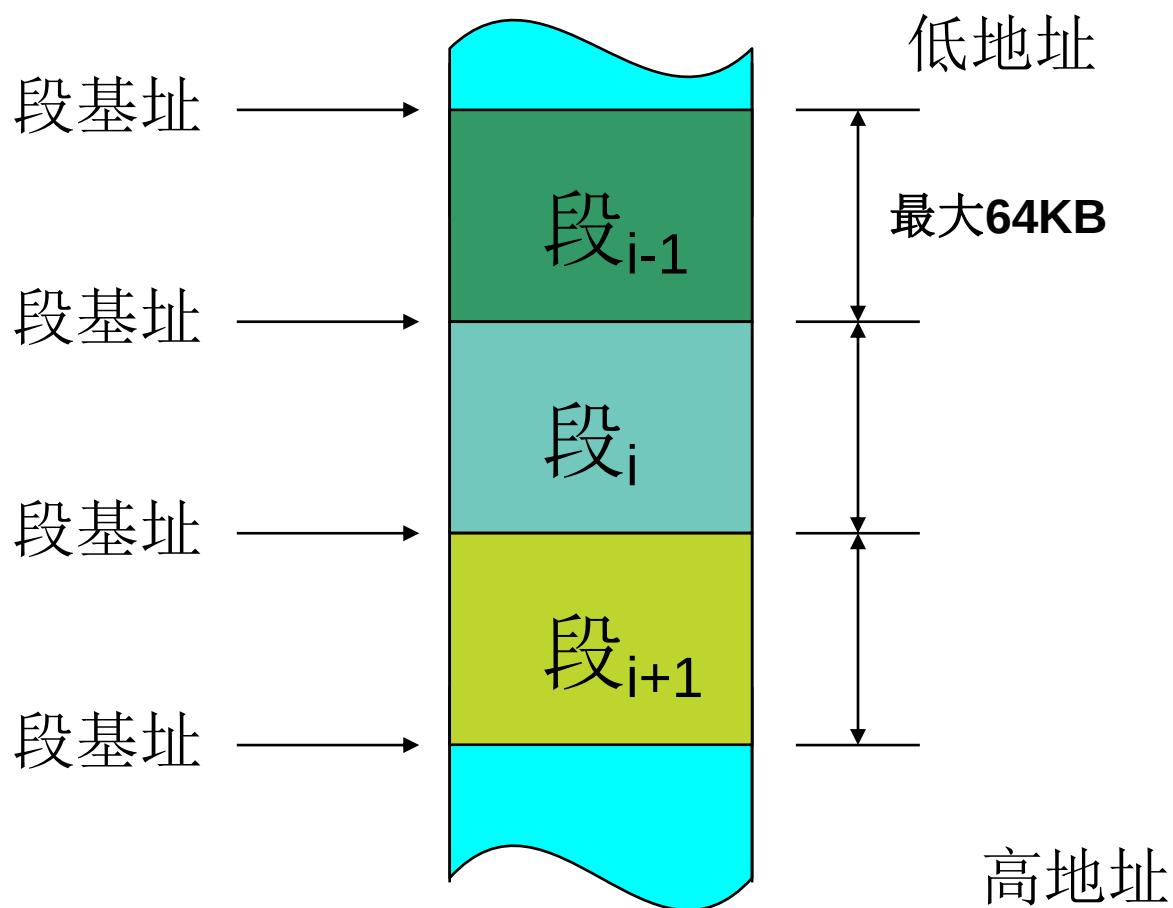
➤解决：存储器分段

物理地址



存储器分段

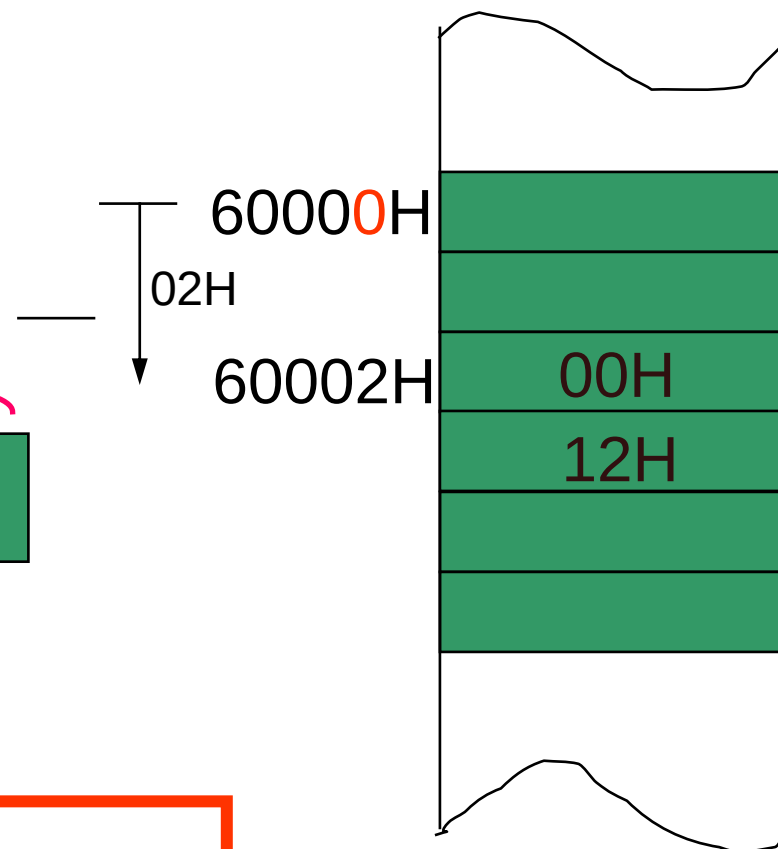
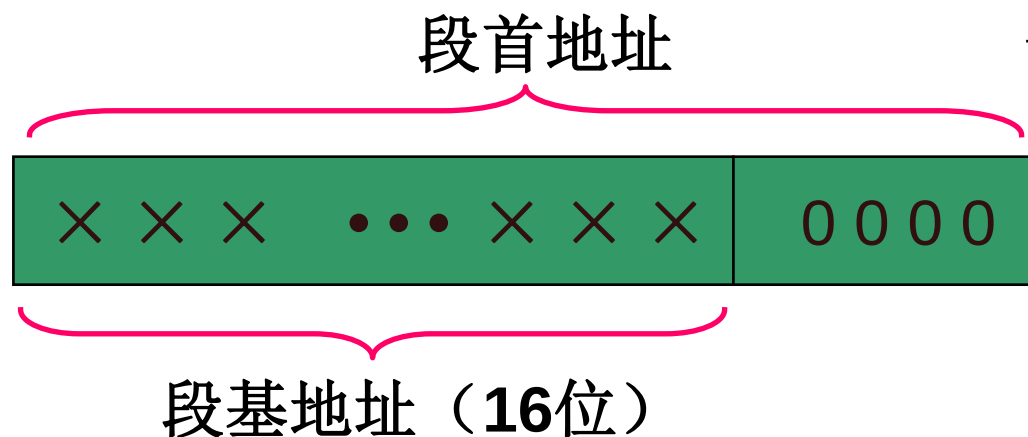
1. 1M存储空间分成若干个逻辑段，每一段 $\leq 64K$
2. 段与段之间可以连续排列，部分重叠，断续排列。



逻辑地址

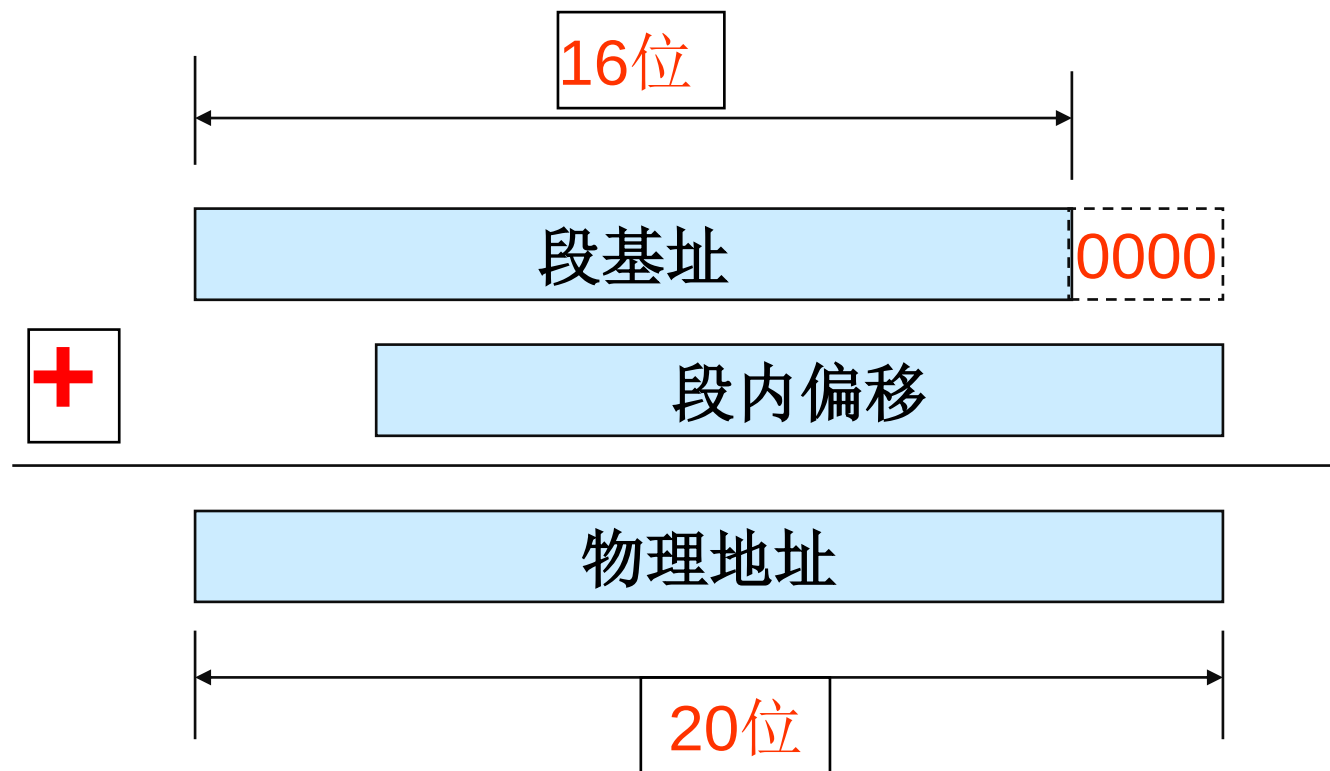
- 段基地址和段内偏移组成了逻辑地址
段地址 偏移地址(偏移量)

格式为: 段地址:偏移地址



物理地址=段基地址 $\times$ 16+偏移地址

- BIU中的地址加法器用来实现逻辑地址到物理地址的变换
- 8088 可同时访问4个段，4个段寄存器指示了每个段的基地址

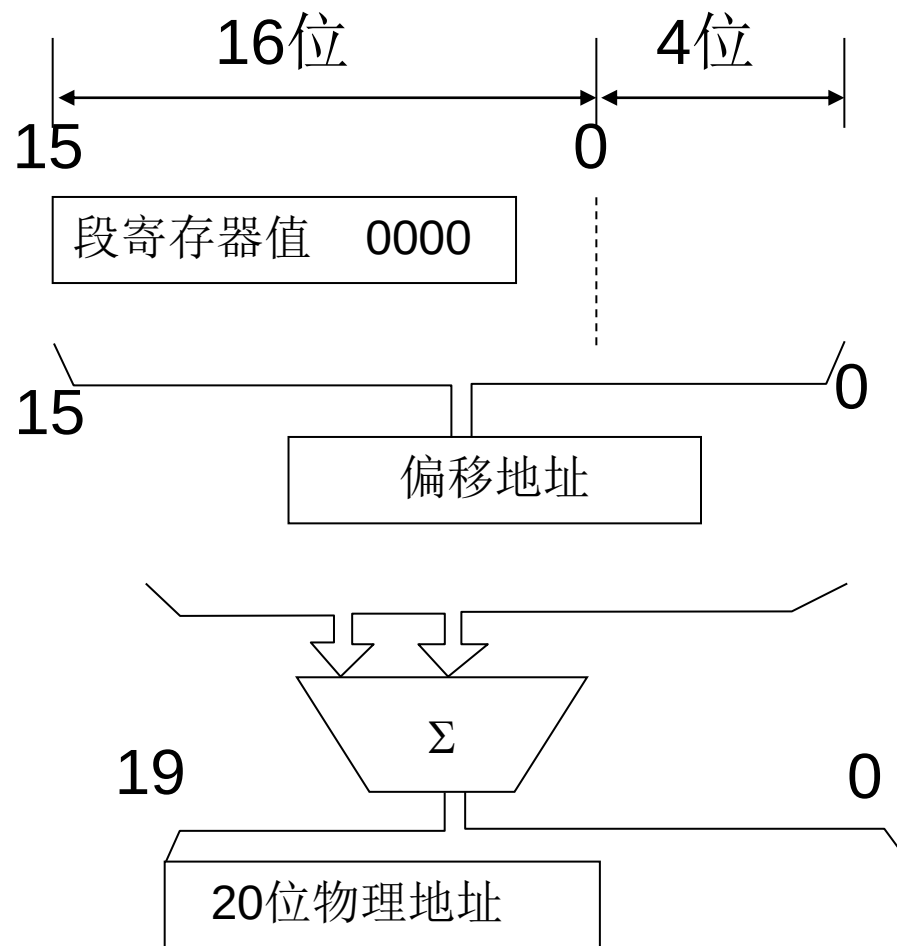


逻辑地址和物理地址区别

物理地址: **20位绝对地址**

逻辑地址: **段基址:段内偏移量**

物理地址 = 段基址 $\times$ 16 + 偏移地址



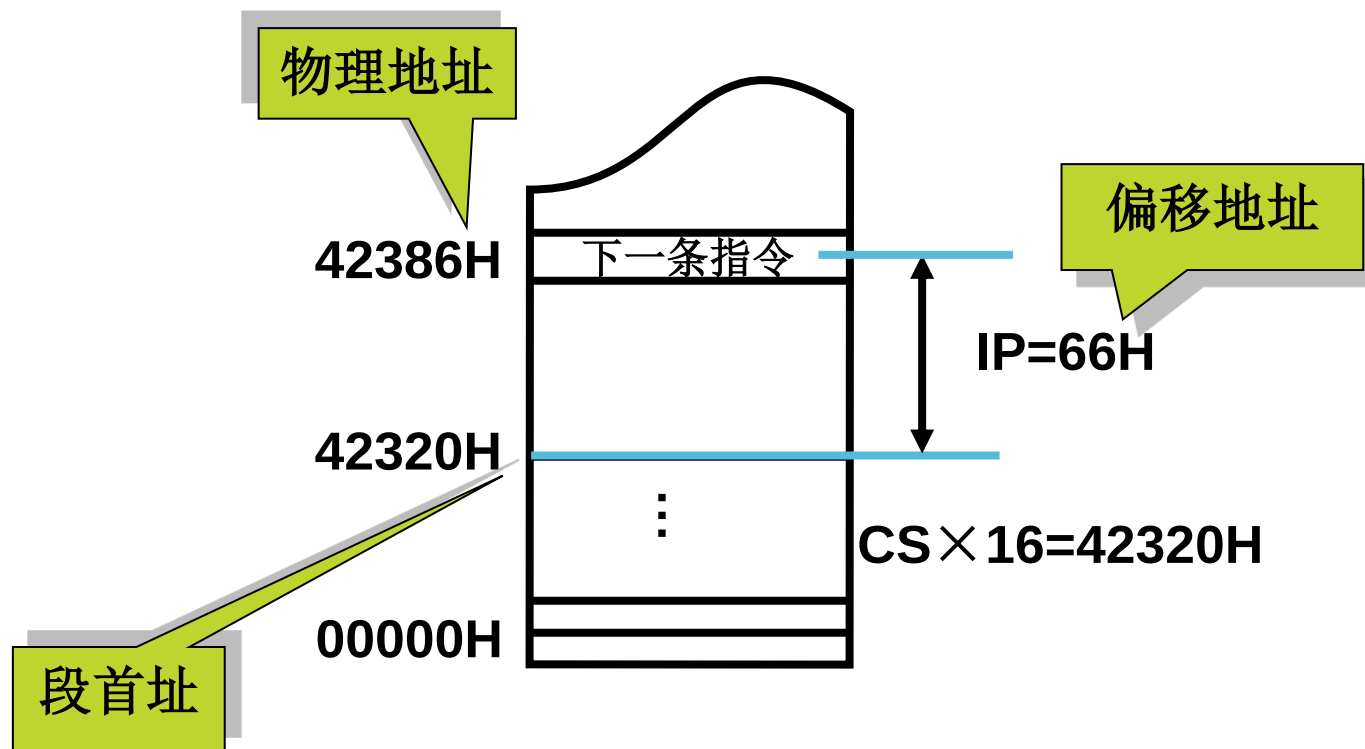
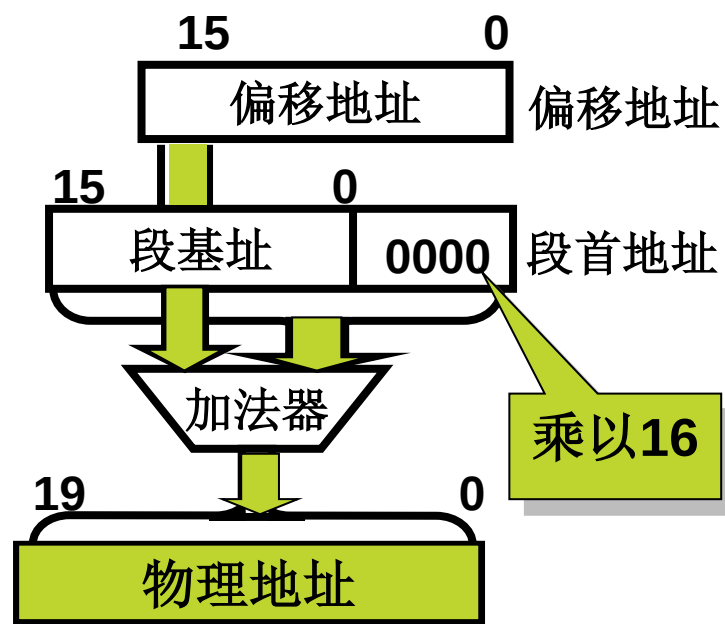
2.2.3 存储器结构 (1)

寻址能力

地址范围

20条地址线 $\rightarrow 2^{20}=1\text{M}$ 字节 $\rightarrow 00000\sim\text{FFFFFFH}$

例：已知IP=66H, CS=4232H



2.2.3 存储器结构 (2)

物理地址: **1MB**存储区域中的实际地址 (**20**位的地址)

物理地址=段地址×**16**+偏移量

逻辑地址: 程序中出现的地址, 由段地址和段内偏移量组成

(注: 段地址和段偏移量都是**16**位的二进制数)

存储器管理就是将程序中的逻辑地址转换成物理地址的结构

•采用段寄存器的优点

*同一物理地址可以有不同的段地址和偏移量

1. 解决了**16**位寄存器如何访问大于**64KB**内存空间的问题
2. 可以实现程序的重定位

内存地址的一般情况

1. 指令地址: $CS \times 16 + IP$

2. 堆栈操作地址: $SS \times 16 + SP$

3. 操作数地址: $DS(ES) \times 16 + \text{偏移地址}$

(其中的偏移地址取决于指令的寻址方式共24种)

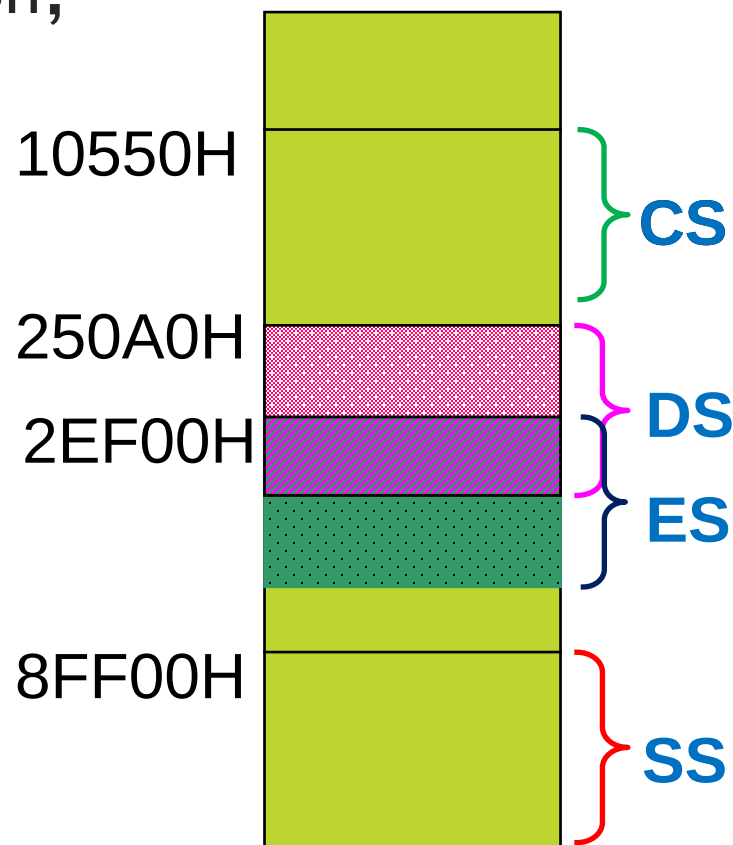
[例]：

- 已知CS=1055H, DS=250AH, ES=2EF0H, SS=8FF0H,
DS段有一操作数, 其偏移地址=0204H,
1) 画出各段在内存中的分布
2) 指出各段首地址
3) 该操作数的物理地址=?

解： 各段分布及段首址见右图所示。

操作数的物理地址为：

$$250AH \times 10H + 0204H = 252A4H$$



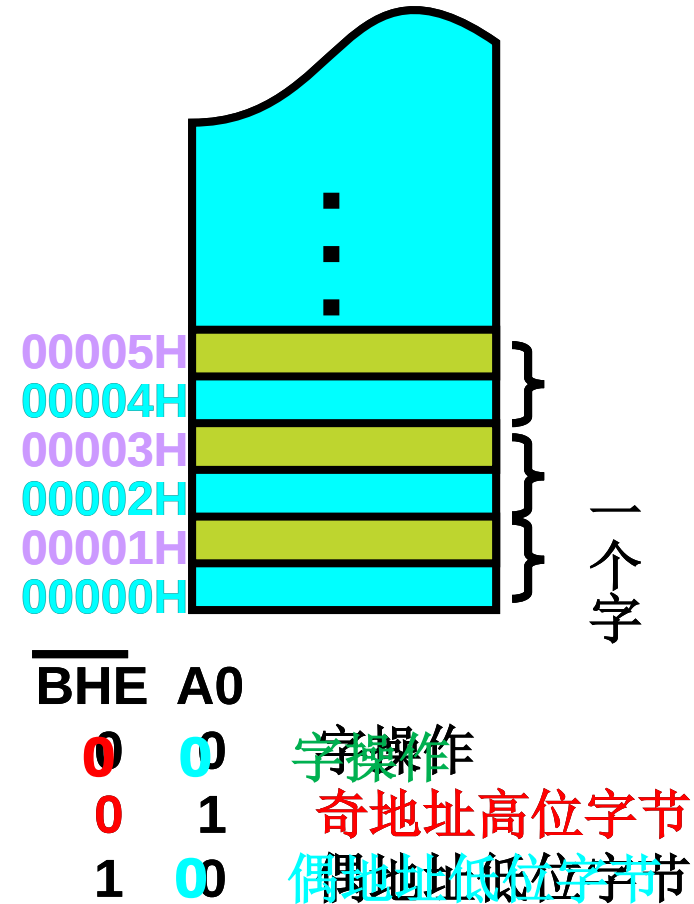
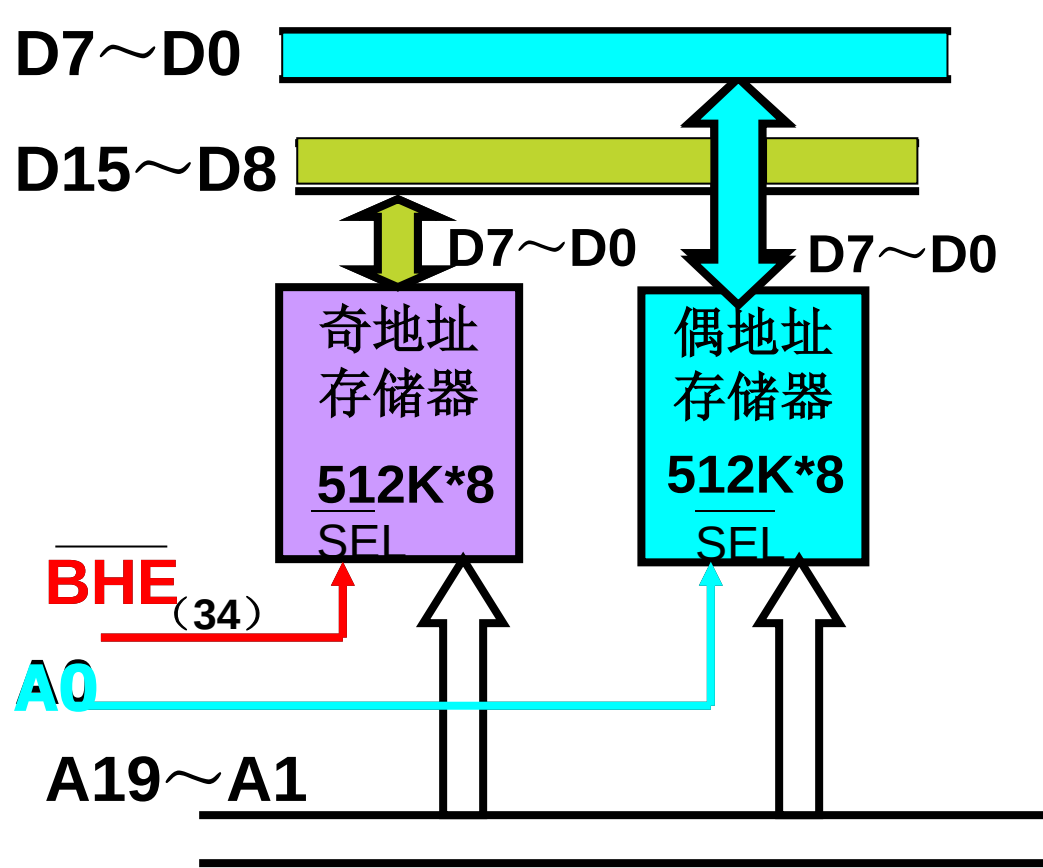
注意：

一个程序可以有代码段、数据段、附加段和堆栈段，它们的段基址分别存放在CS、DS、ES和SS中。段寄存器为16位，在计算物理地址时要乘16，比如DS的内容为1000H，对应物理地址为10000H，相当于在二进制数尾部补了4个0，也就是说，段的起始地址只能从物理地址能够整除16的（二进制数尾部为4个0）那些地址开始。

例如，如下第一列的物理地址可以作为段首地址，存入段寄存器时只存前16位二进制数，其他列不可以作为段首地址。

00000H	00001H	00002H	00003H	...	0000FH
00010H	00011H	00012H	00013H	...	0001FH
00020H	00021H	00022H	00023H	...	0002FH
00030H	00031H	00032H	00033H	...	0003FH

8086的存储器的分体结构

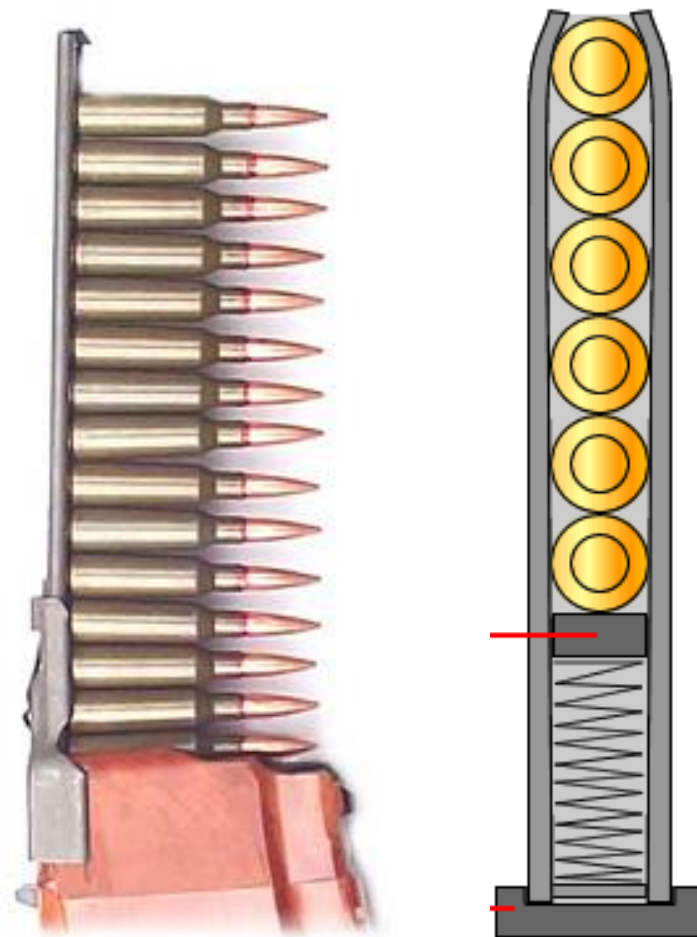


四、堆栈及堆栈段的使用

- 内存中一个按LIFO方式操作的特殊区域
- 每次压栈和退栈均以WORD为单位
- SS存放堆栈段地址，SP存放段内偏移，SS:SP构成了堆栈指针
- 堆栈用于存放返回地址、过程参数或需要保护的数据
- 常用于响应中断或子程序调用

堆栈的概念

- 堆栈区是在随机存储的内存中开辟一片数据存储区实现FILO (或LIFO) 的数据存放。
- 这种数据存取方式类似于货物的堆放与取出，故称作堆栈。也类似于弹匣中子弹的存取方式。
- 堆栈的FILO存取方式是靠堆栈指针SP的自动变化来实现的。



例1:入栈指令 **PUSH**

$$SS \times 16 + SP = 02008H$$

PUSH CX

若已知 $SS=0200H$,
 $SP=0008H$,
 $CX=12FAH$,
操作示意图如图所示:

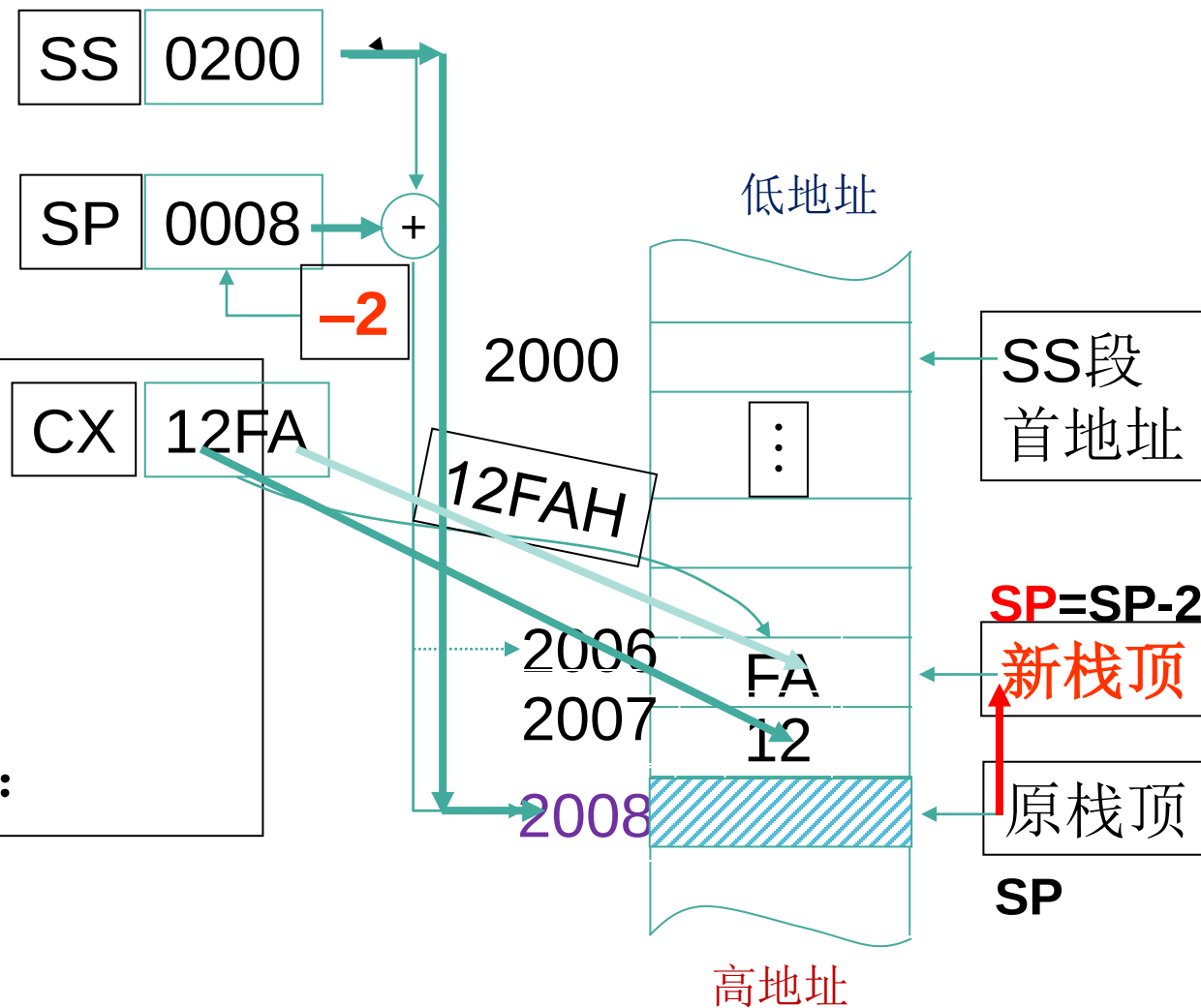


图 PUSH CX 的操作过程

例2:出栈指令POP

$$SS \times 16 + SP = 02006H$$

POP CX

操作示意图如图所示:

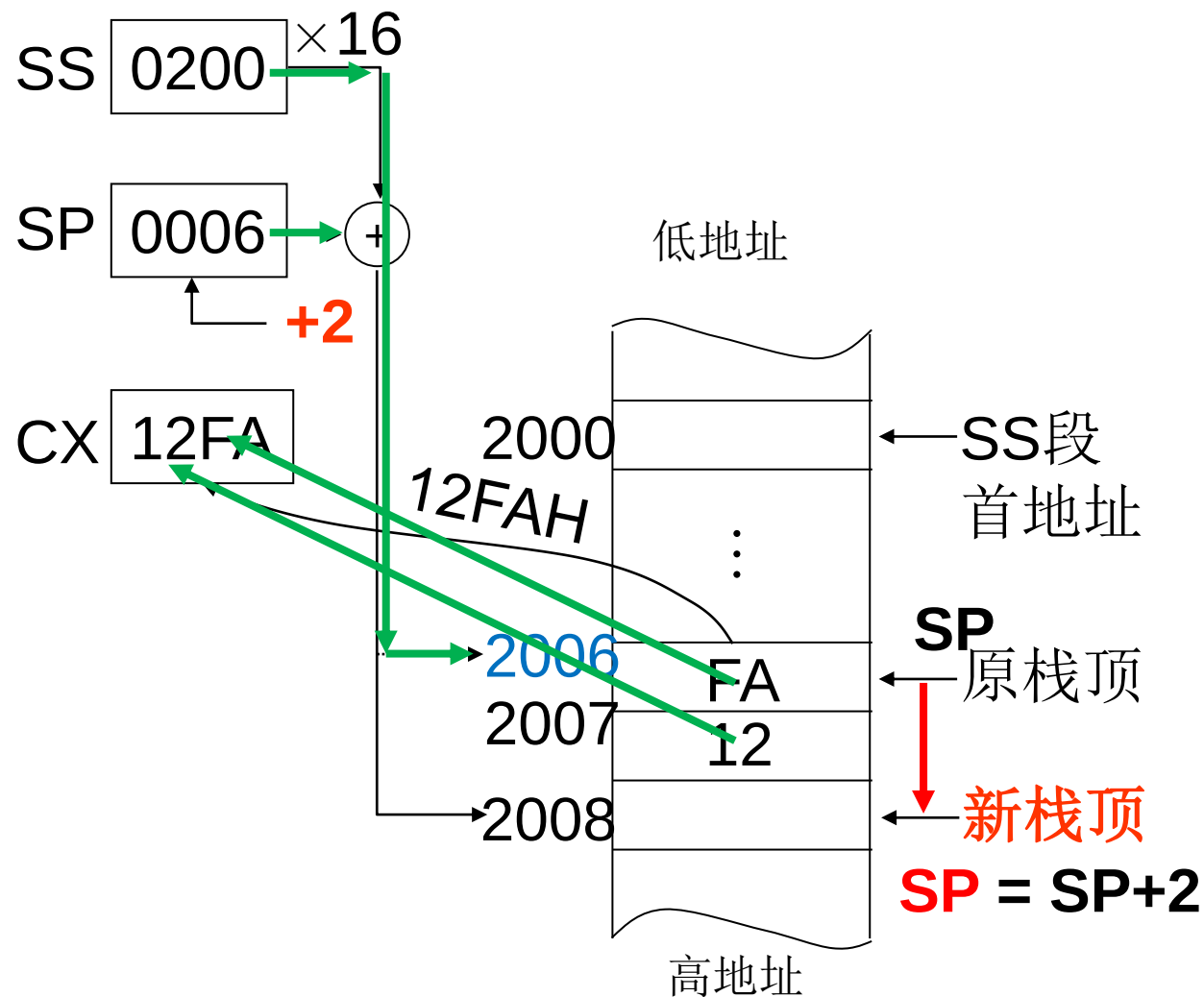
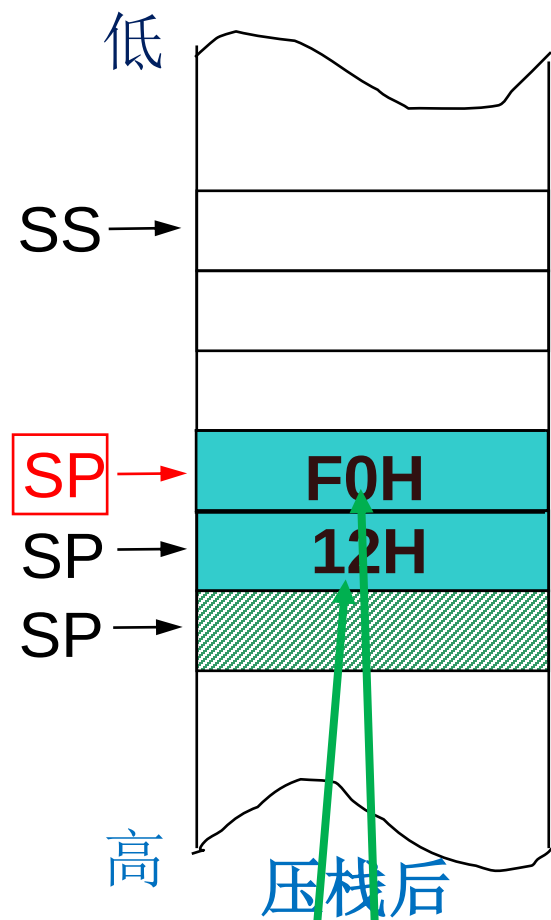
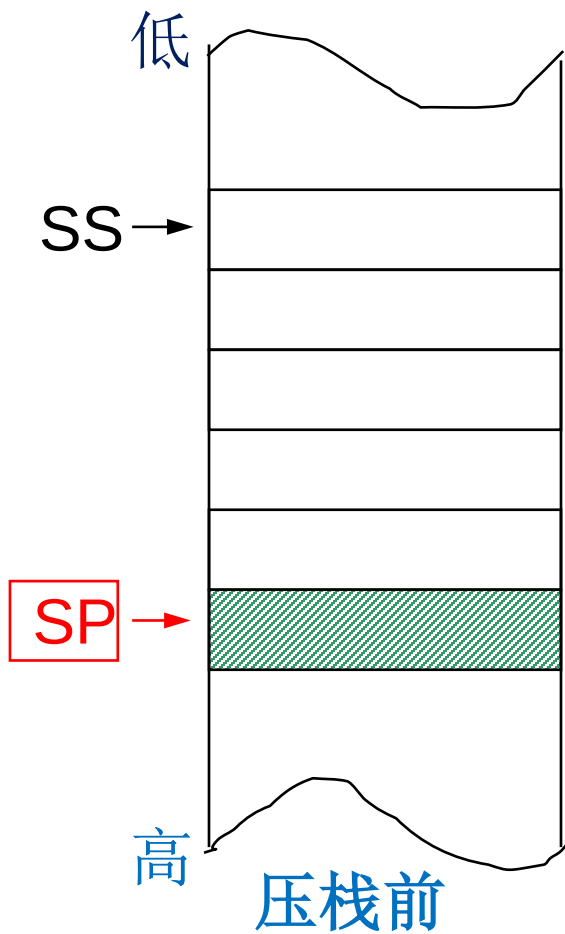
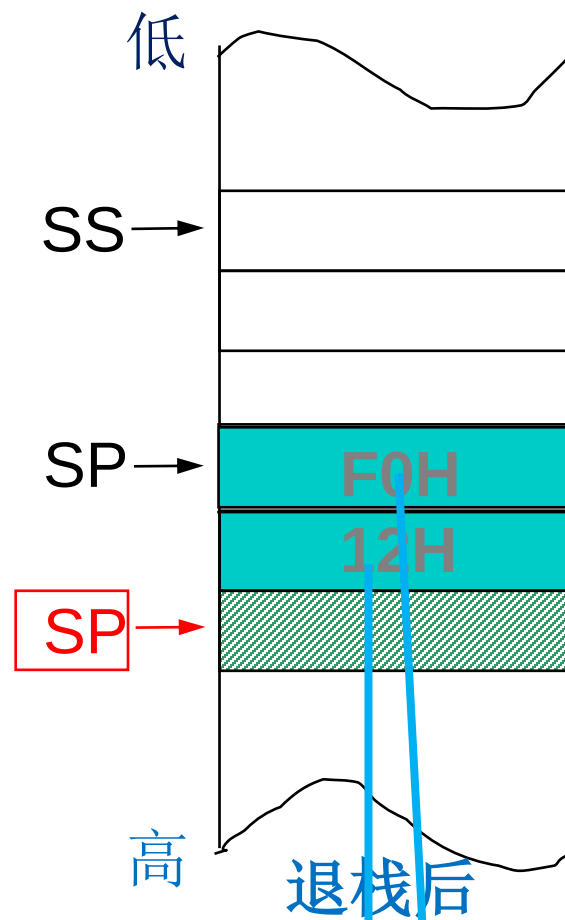


图 POP CX的操作过程



AX=12F0H
PUSH AX



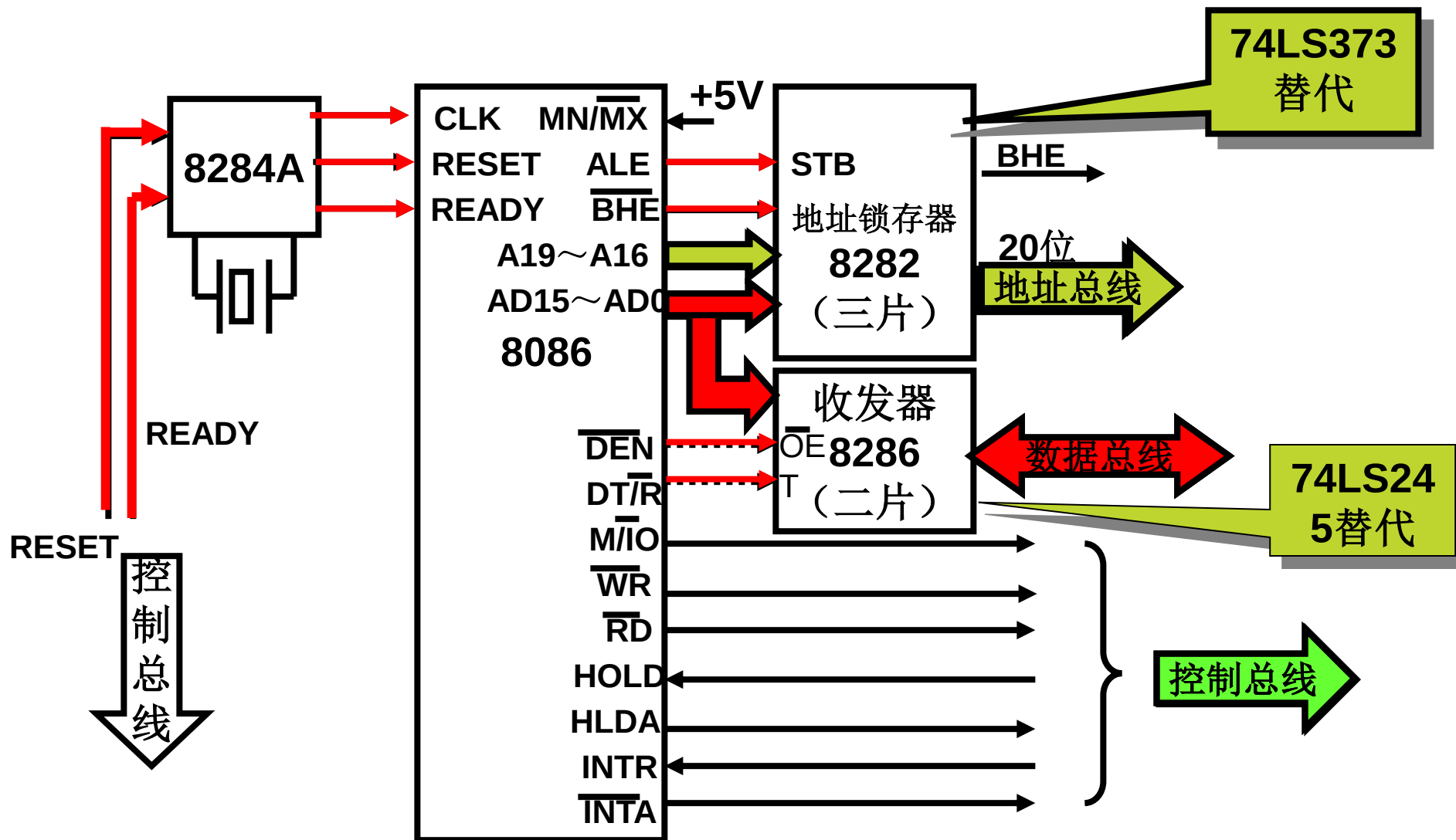
POP BX
BX=12F0H



2.4 8086微处理器的工作模式



•最小工作模式下的典型配置



常用接口芯片介绍

➤ 8282/74LS373 —— 具有三态正相输出的锁存器

➤ 内部包含8个D触发器

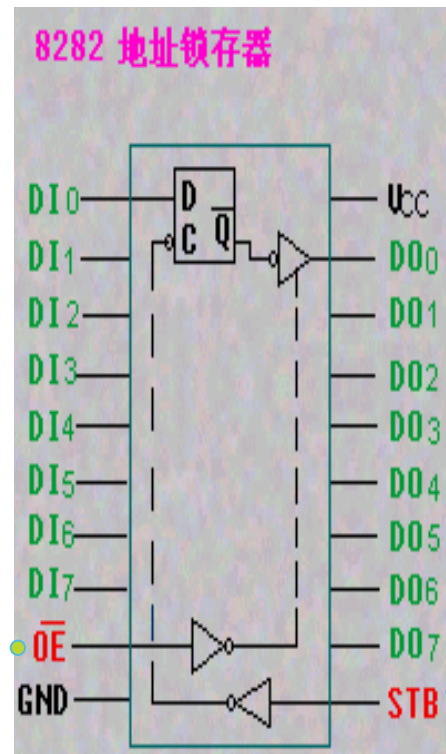
◆ 引脚:

- $DI_0 \sim DI_7$ 输入
- $DO_0 \sim DO_7$ 输出
- STB 锁存信号
- $OE\#$ 输出允许

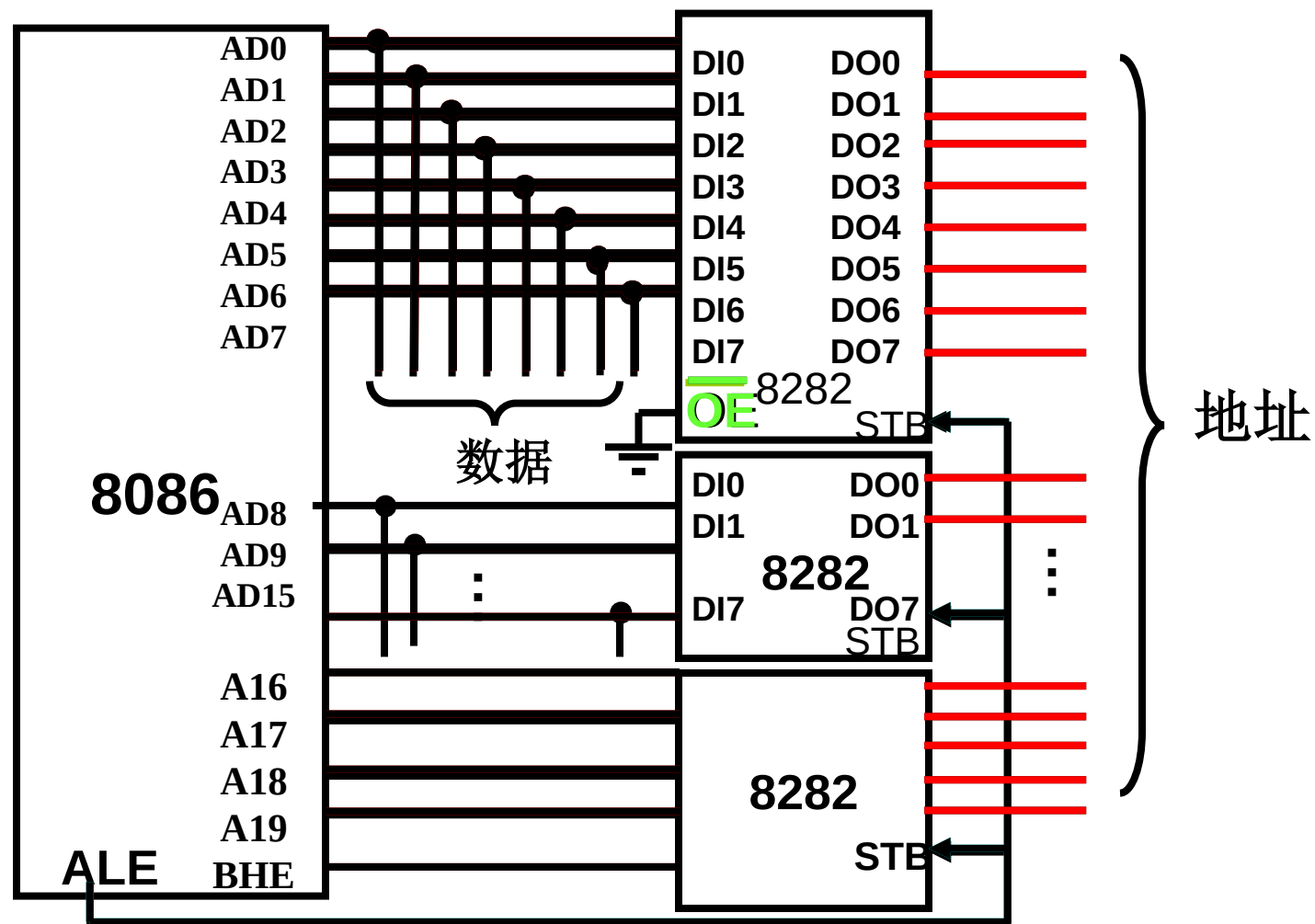
➤ 功能:

- STB = 1 锁存数据
- $OE\# = 0$ 将锁存的数据输出

➤ 功能类似的还有8283——但为反相输出

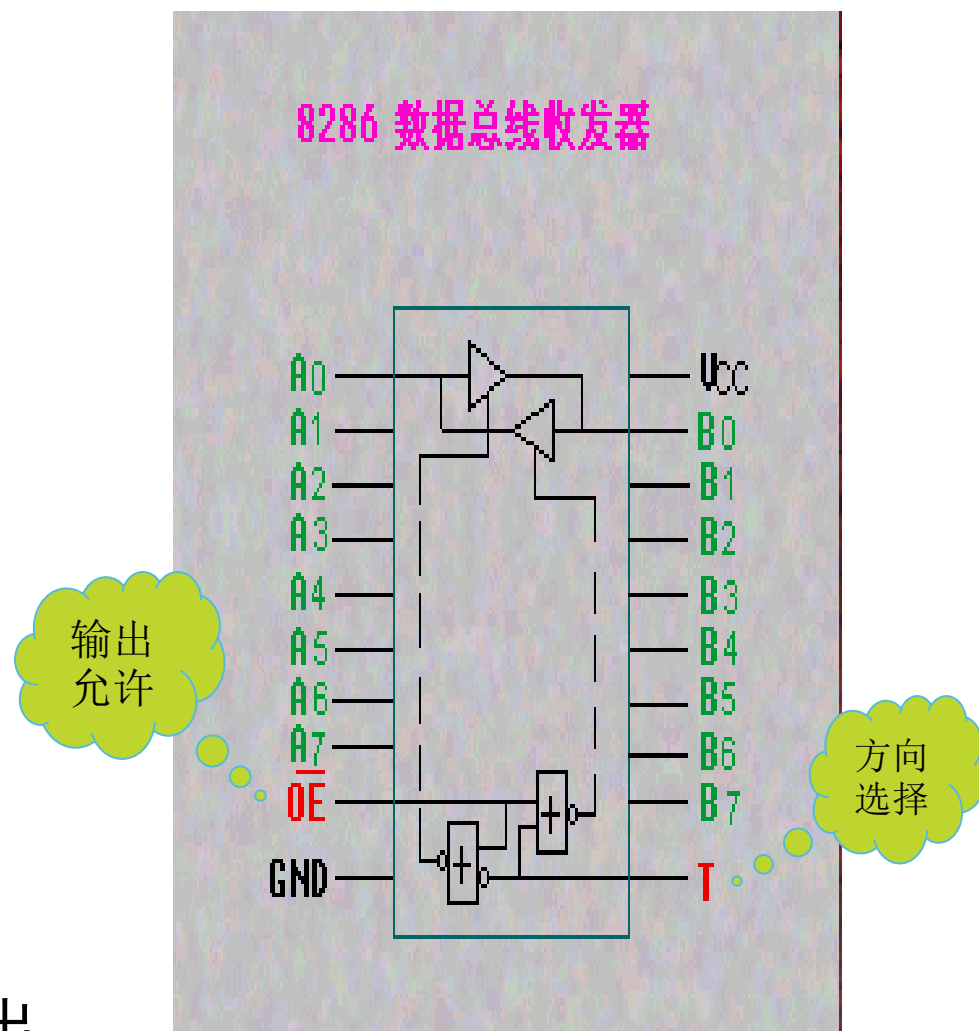


8282锁存器与8086的连接

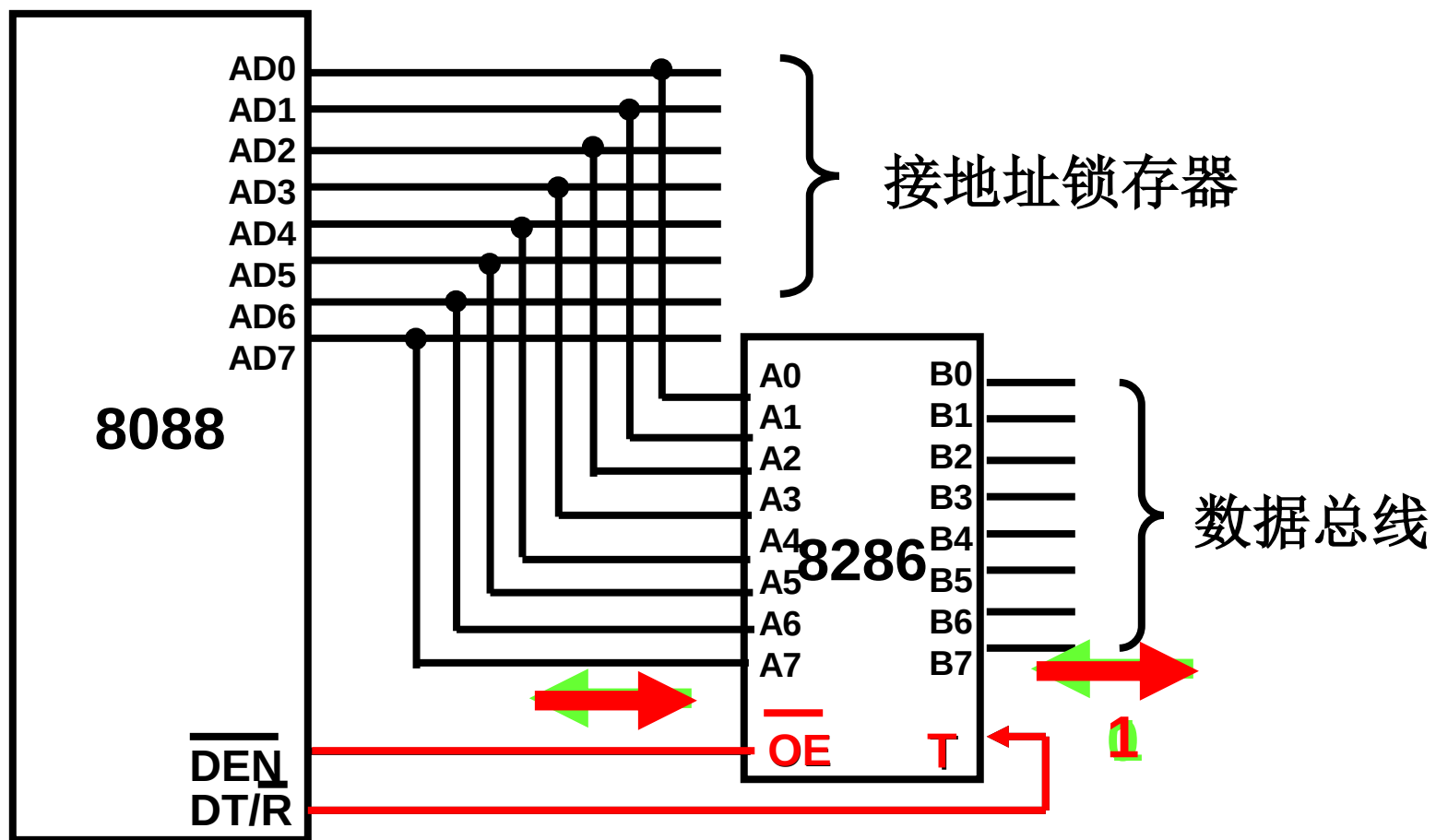


总线驱动器

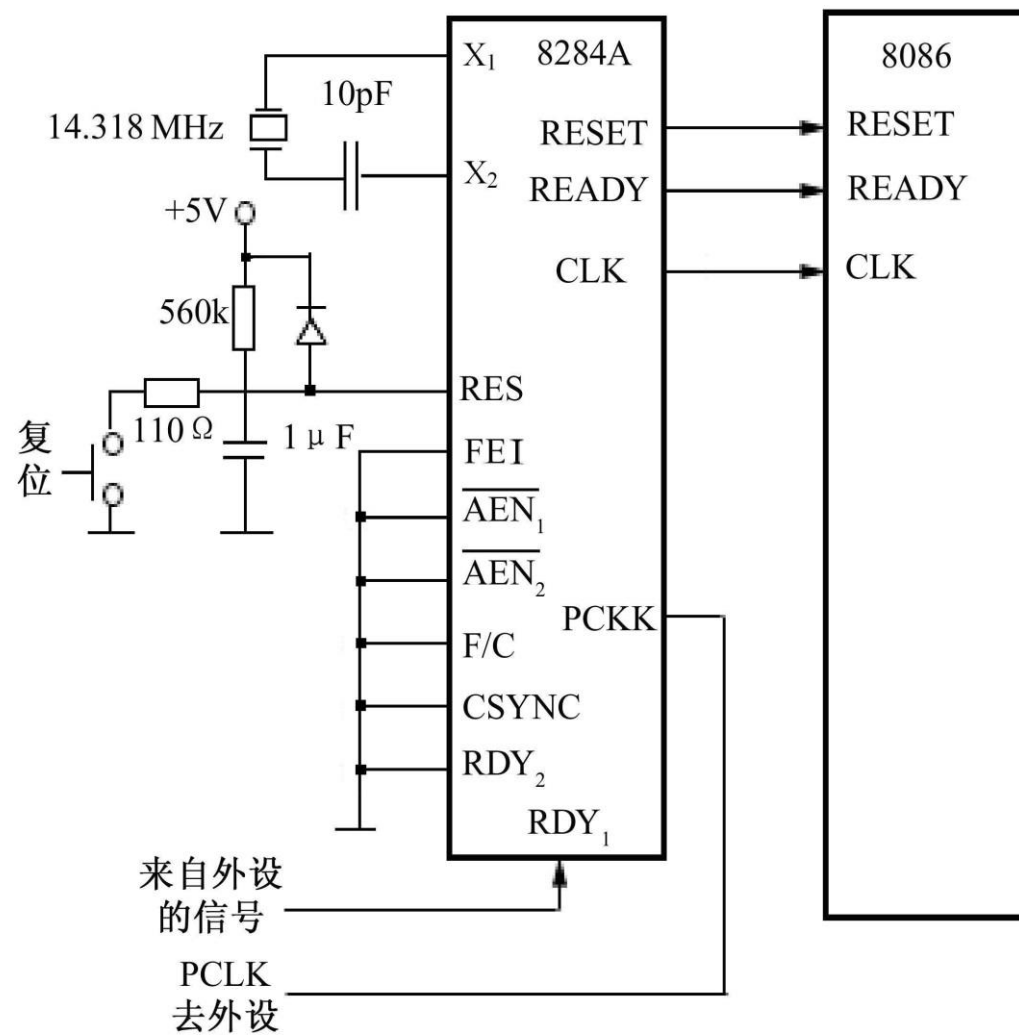
- 8286 / 74LS245 —— 双向三态驱动器引脚
 - $A_0 \sim A_7$ 和 $B_0 \sim B_7$ 双向数据线
 - $OE\#$ 输出允许
 - T 方向控制
- 功能：
 - $OE\#=0$ 时，门导通；
 - 门导通时： $T=0$ ， $B \rightarrow A$ ； $T=1$ ， $A \rightarrow B$
- 功能类似的还有 8287 —— 但为反相输出



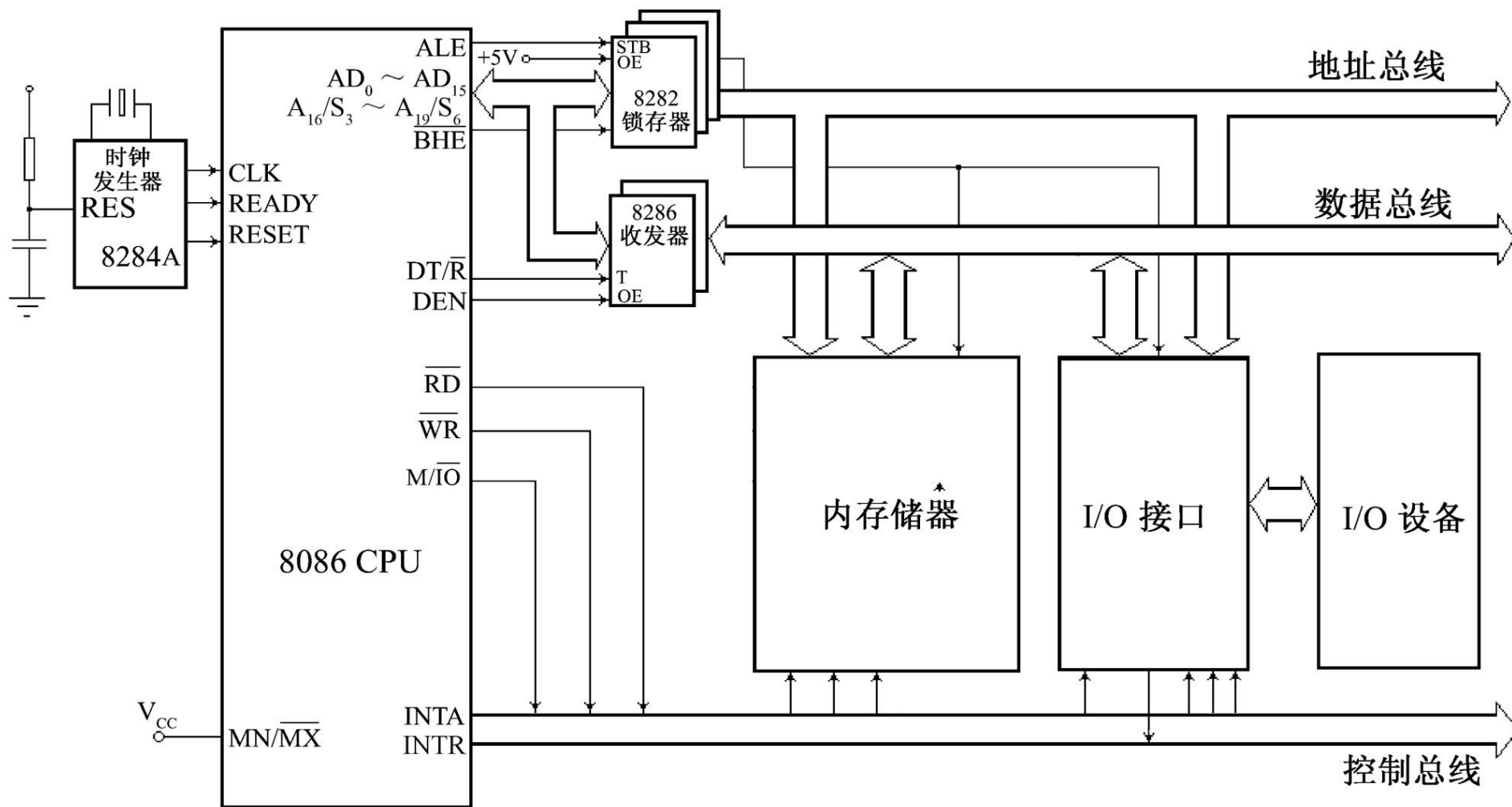
- 8286收发器和8088的连接



8284A与8086的连接图



最小模式的系统组成



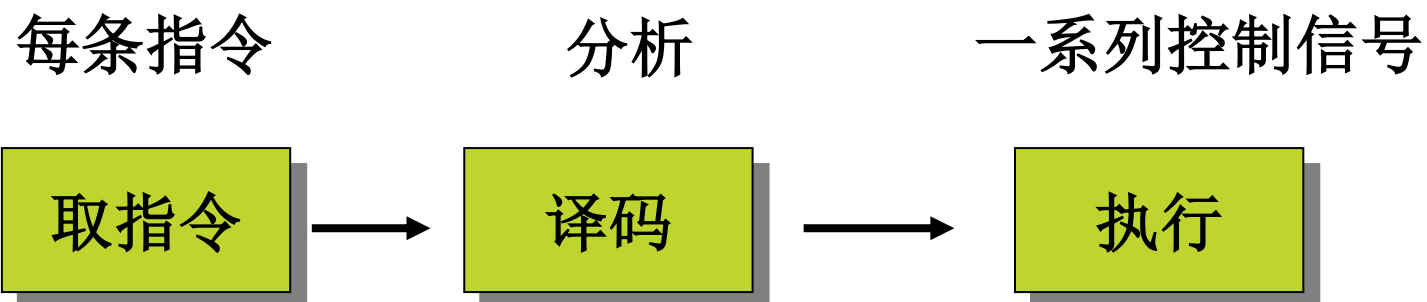
8086 最小模式系统组成框图



2.5 8086/8088的总线操作时序



2.4 8086总线的工作周期（1）



指令周期：执行一条指令所需要的时间

执行每一条指令的时间不同

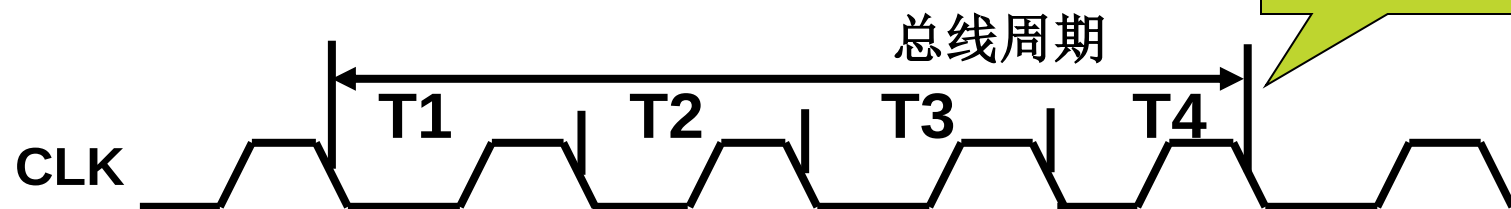
总线周期：**CPU**从存储器或**I/O**端口存取一次所需要的时间（**Bus Cycle**）

访问一次总线的时间

2.4

8086总线的工作周期（2）

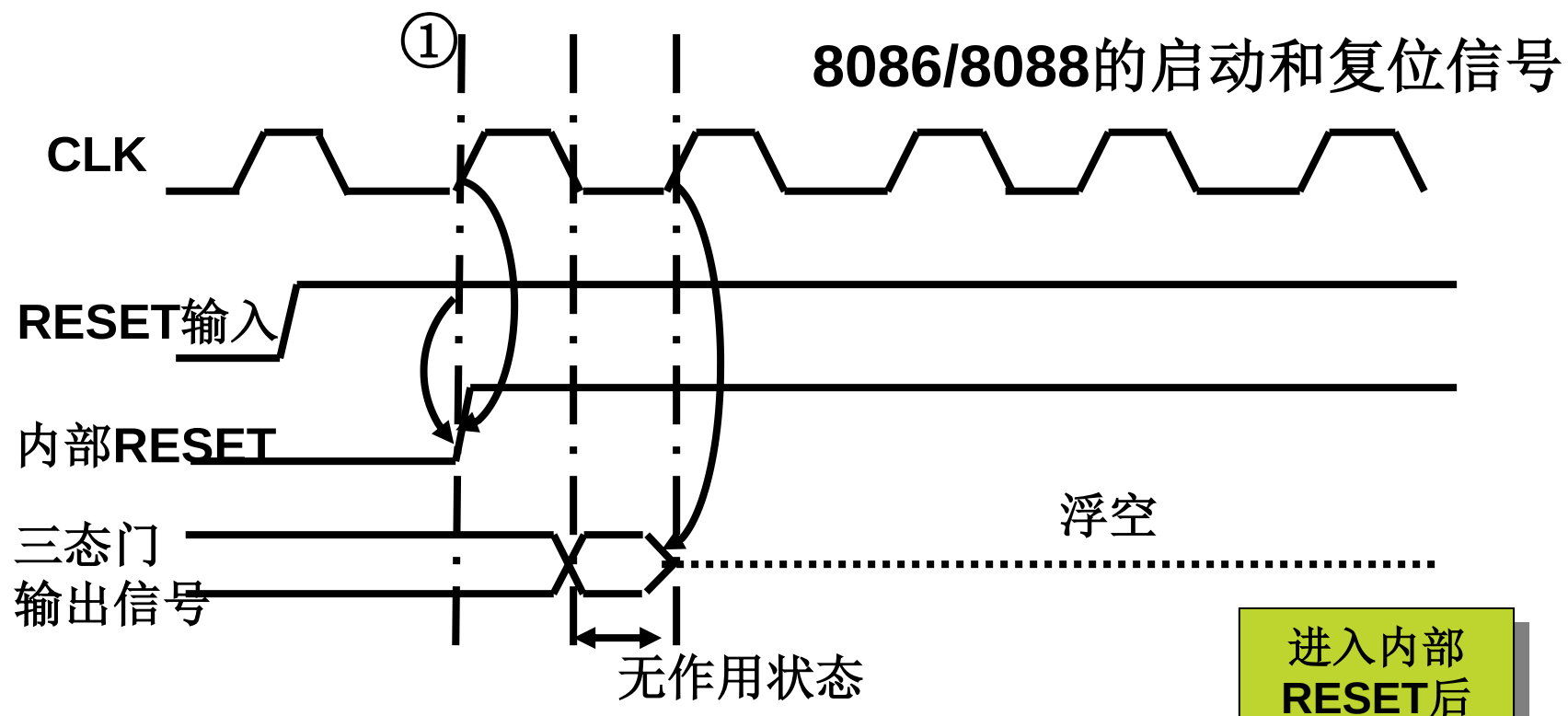
每个总线周期通常包含4个T状态



8086的时钟频率为10MHZ，时钟周期（一个T状态）为100ns

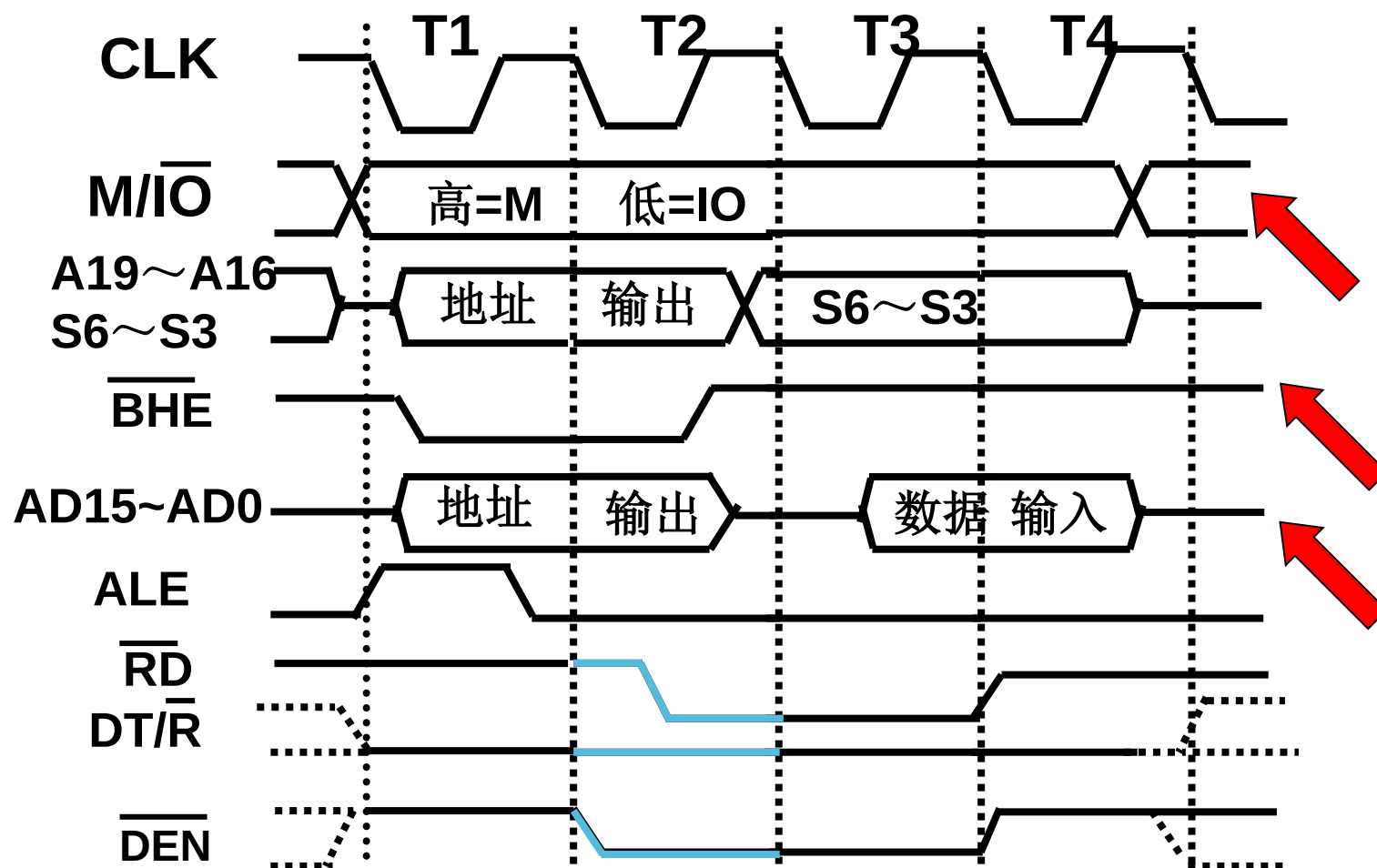
基本总线周期 { 存储器读或写
输入输出的读或写
中断响应

2.4.1 系统的复位和启动操作

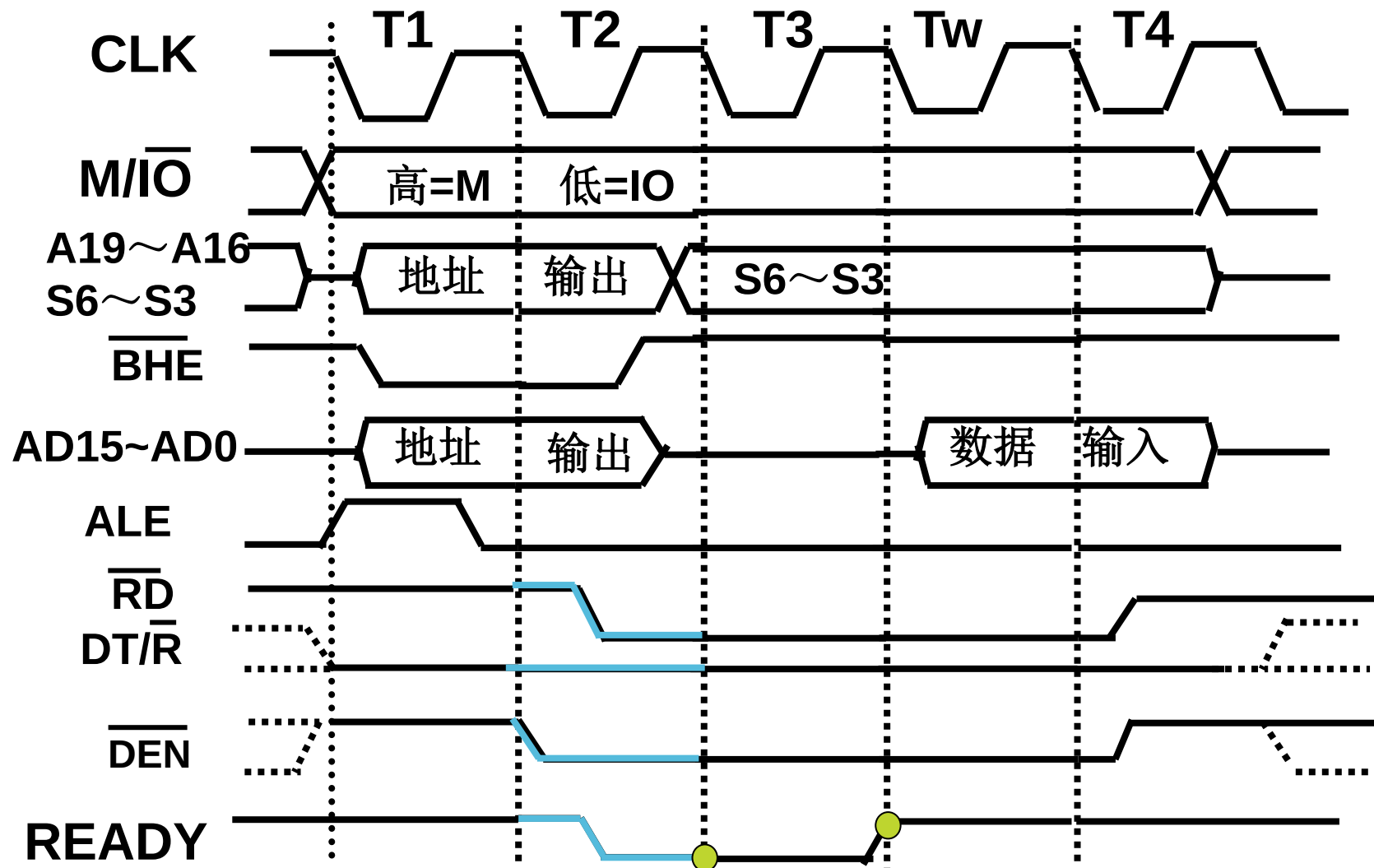


CPU立即停止操作，清FR，DS，ES，SS、IP及指令队列。
同时，置CS为FFFFH。当RESET变为低电平时，CPU从FFFF0H单元开始启动。

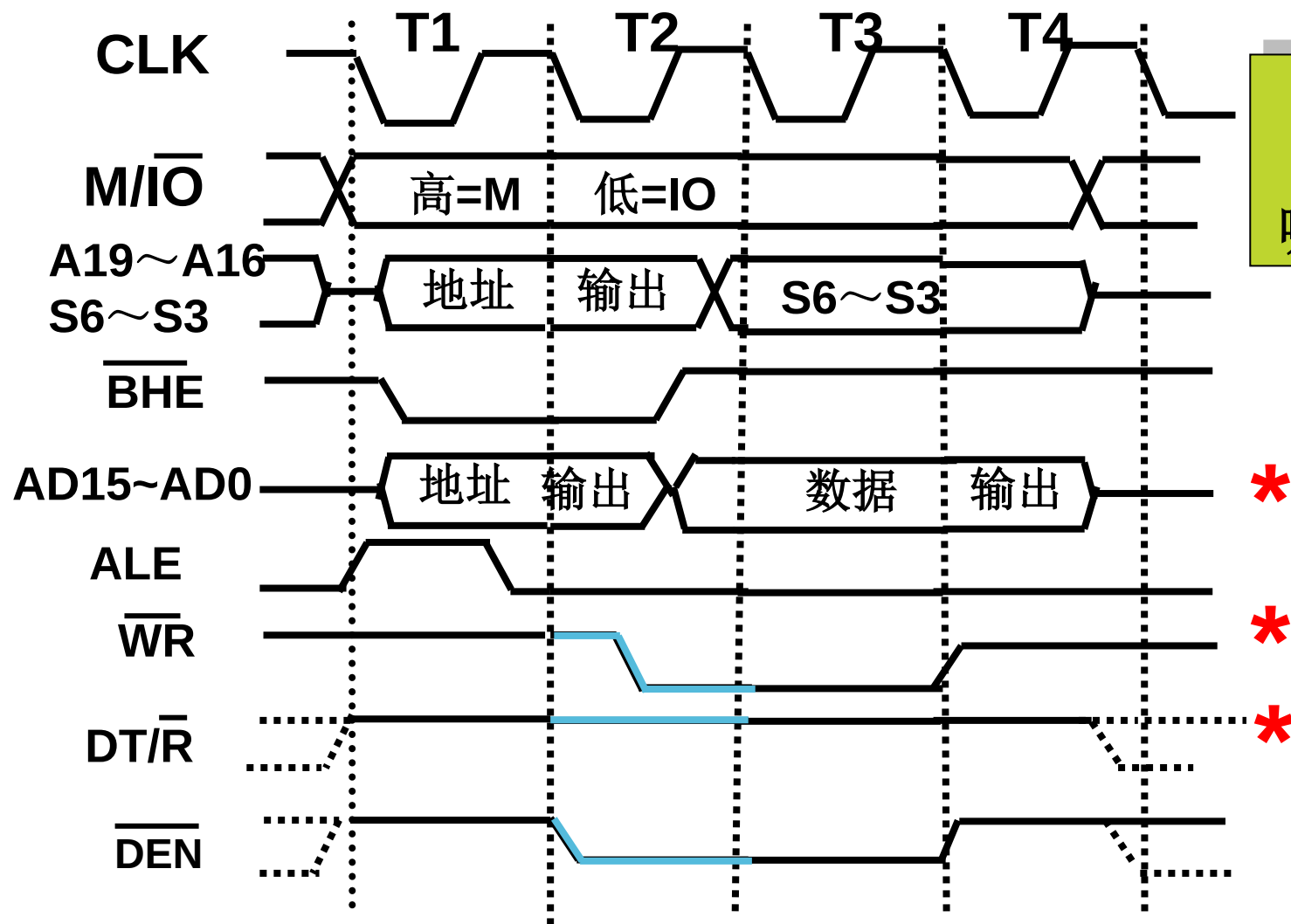
2.4.2 最小工作模式下的总线读周期



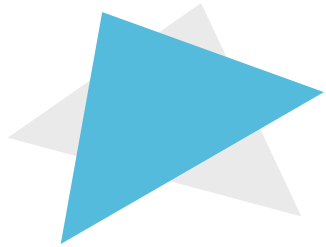
• 具有等待状态的总线读操作



2.4.4 最小工作模式下的总线写操作 (8086)



读操作和
写操作有
哪些不同?



The end, thank you!